[NOME DA INSTITUIÇÃO DE ENSINO]
[NOME DA FACULDADE OU DEPARTAMENTO]
[NOME DO CURSO]

RAFAEL LOURENÇO

ROBOFAEL: assistente pessoal com avatar animado e interação por voz, construído com arquitetura hexagonal e princípios SOLID

[Cidade]
2026

RAFAEL LOURENÇO

ROBOFAEL: assistente pessoal com avatar animado e interação por voz, construído com arquitetura hexagonal e princípios SOLID

Trabalho de Conclusão de Curso apresentado ao [NOME DO CURSO] da [NOME DA INSTITUIÇÃO DE ENSINO], como requisito parcial para a obtenção do título de [bacharel em NOME DO CURSO].

Orientador(a): [Nome completo do(a) orientador(a)].

[Cidade]

2026

RAFAEL LOURENÇO

ROBOFAEL: assistente pessoal com avatar animado e interação por voz, construído com arquitetura hexagonal e princípios SOLID

Trabalho de Conclusão de Curso apresentado ao [NOME DO CURSO] da [NOME DA INSTITUIÇÃO DE ENSINO], como requisito parcial para a obtenção do título de [bacharel em NOME DO CURSO].

Aprovado em: _____ / _____ / _____.

BANCA EXAMINADORA

_____

[Nome completo do(a) orientador(a)] (Orientador(a))
[NOME DA INSTITUIÇÃO DE ENSINO]

_____

[Nome completo do(a) avaliador(a)]
[Instituição]

_____

[Nome completo do(a) avaliador(a)]
[Instituição]

RESUMO

Este trabalho apresenta o projeto, a construção e a validação do Robofael, um assistente pessoal de atividades diárias para computadores com Linux que conversa por voz e é representado por um avatar animado gerado a partir de fotografias do próprio usuário. O sistema ouve comandos falados ou digitados, responde com voz neural ajustada ao tom e ao ritmo da voz do usuário, gerencia tarefas e lembretes, mantém uma memória de longo prazo auditável capaz de responder o que foi feito em um dia ou em uma semana, abre programas instalados e funciona em português e em inglês. A ativação sem as mãos ocorre pela frase “ok Fael”, reconhecida localmente, de modo que o áudio do ambiente não deixa o computador. Quando disponível, um modelo de linguagem de grande porte (Claude) interpreta pedidos em linguagem natural e opera a agenda por meio de ferramentas; sem ele, um modelo local servido pelo Ollama conversa com ferramentas restritas e, sem nenhum dos dois, um interpretador de intenções local mantém as funções essenciais. O desenvolvimento seguiu um processo iterativo e incremental, conduzido por pedidos sucessivos e apoiado por um agente de programação baseado em inteligência artificial, em vinte incrementos registrados em versionamento semântico. A solução adota arquitetura hexagonal, pipeline de fala em fluxo contínuo, máquina de estados explícita, barramento de eventos para isolar a interface gráfica e persistência em SQLite com histórico somente de acréscimo protegido por cadeia de resumos criptográficos SHA-256. Ao final, o código foi revisado de ponta a ponta à luz dos princípios SOLID e das práticas de código limpo: responsabilidades foram separadas, despachos condicionais foram convertidos em tabelas extensíveis e as dependências passaram a apontar apenas para contratos. As regras de arquitetura tornaram-se verificáveis por testes automatizados. Um incremento guiado por medições reduziu o consumo de processamento em repouso de 29% para 5% de um núcleo, levou a janela a aparecer em 0,6 s e eliminou pausas artificiais entre as frases faladas. A validação contou com 643 testes automatizados, cobertura de 87% das linhas e ramificações, verificação estática de tipos e de estilo, complexidade ciclomática máxima igual a 10 por função e um instalador para Ubuntu testado em ambiente isolado. Os resultados indicam que é viável construir um assistente de voz responsivo, privado e de fácil manutenção com componentes livres e serviços de linguagem, e que a verificação automática das regras de projeto preserva a qualidade da arquitetura ao longo da evolução do software.

Palavras-chave: assistente virtual; reconhecimento de fala; arquitetura hexagonal; princípios SOLID; código limpo.

ABSTRACT

This work presents the design, construction and validation of Robofael, a personal daily-activity assistant for Linux computers that talks by voice and is represented by an animated avatar generated from photographs of its own user. The system listens to spoken or typed commands, answers with a neural voice tuned to the pitch and pace of the user's voice, manages tasks and reminders, keeps an auditable long-term memory able to answer what was done on a given day or week, opens installed programs and works in Portuguese and English. Hands-free activation happens through the phrase "ok Fael", recognized locally, so ambient audio never leaves the computer. When available, a large language model (Claude) interprets natural-language requests and operates the agenda through tools; without it, a local model served by Ollama talks with restricted tools and, without either, a local intent interpreter keeps the essential functions. Development followed an iterative and incremental process, driven by successive requests and supported by an artificial-intelligence coding agent, in twenty increments tracked with semantic versioning. The solution adopts a hexagonal architecture, a streaming speech pipeline, an explicit state machine, an event bus that isolates the graphical interface, and SQLite persistence with an append-only history protected by a SHA-256 hash chain. Finally, the code was reviewed end to end in the light of the SOLID principles and clean-code practices: responsibilities were separated, conditional dispatch was replaced with extensible tables, dependencies were made to point only to contracts, and architecture rules became verifiable through automated tests. A measurement-driven increment cut idle processor usage from 29% to 5% of one core, made the window appear in 0.6 s and removed artificial pauses between spoken sentences. Validation relied on 643 automated tests, 87% line and branch coverage, static type and style checking, a maximum cyclomatic complexity of 10 per function, and an Ubuntu installer tested in an isolated environment. The results indicate that it is feasible to build a responsive, private and maintainable voice assistant from free components and language services, and that automatically checking design rules preserves architectural quality as the software evolves.

Keywords: virtual assistant; speech recognition; hexagonal architecture; SOLID principles; clean code.

LISTA DE QUADROS

Quadro 1 – Tecnologias e ferramentas utilizadas 22
Quadro 2 – Requisitos funcionais de interação e avatar 24
Quadro 3 – Requisitos funcionais de agenda e memória 25
Quadro 4 – Requisitos funcionais de inteligência e integração com o sistema 26
Quadro 5 – Requisitos funcionais de configuração, idioma e distribuição 27
Quadro 6 – Requisitos funcionais incorporados nas versões 3.6.0 a 3.8.0 27
Quadro 7 – Requisitos não funcionais de desempenho, confiabilidade e segurança . 28
Quadro 8 – Requisitos não funcionais de usabilidade, manutenibilidade e portabilidade 29
Quadro 9 – Camadas da arquitetura, módulos e dependências permitidas 32
Quadro 10 – Tabelas do banco de dados 34
Quadro 11 – Principais decisões de projeto 39
Quadro 12 – Suítes de teste automatizado 60
Quadro A.1 – Rastreabilidade dos requisitos funcionais 75
Quadro A.2 – Rastreabilidade dos requisitos não funcionais 75
Quadro B.1 – Exemplos de comandos de voz 77
Quadro C.1 – Variáveis de ambiente e opções de linha de comando 78

LISTA DE TABELAS

Tabela 1 – Evolução do sistema por incremento 41
Tabela 2 – Métricas antes e depois da revisão 58
Tabela 3 – Cobertura de testes por camada (linhas e ramificações) 62
Tabela 4 – Medições de desempenho 65
Tabela 5 – Medições de desempenho e fluidez: versões 3.3.0 e 3.4.0 66

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
AGC	*Automatic Gain Control* (controle automático de ganho)
API	*Application Programming Interface* (interface de programação de aplicações)
CI	*Continuous Integration* (integração contínua)
DIP	*Dependency Inversion Principle* (princípio da inversão de dependência)
DSP	*Digital Signal Processing* (processamento digital de sinais)
FPS	Quadros por segundo
GIL	*Global Interpreter Lock* do interpretador CPython
GUI	*Graphical User Interface* (interface gráfica do usuário)
ISP	*Interface Segregation Principle* (princípio da segregação de interfaces)
JSON	*JavaScript Object Notation*
LLM	*Large Language Model* (modelo de linguagem de grande porte)
LRU	*Least Recently Used* (menos recentemente usado)
LSP	*Liskov Substitution Principle* (princípio da substituição de Liskov)
OCP	*Open-Closed Principle* (princípio aberto/fechado)
PCM	*Pulse-Code Modulation* (modulação por código de pulso)
RF	Requisito funcional
RNF	Requisito não funcional
SHA	*Secure Hash Algorithm*
SOLID	Acrônimo dos cinco princípios de projeto orientado a objetos (SRP, OCP, LSP, ISP, DIP)
SQL	*Structured Query Language*
SRP	*Single Responsibility Principle* (princípio da responsabilidade única)
STT	*Speech-to-Text* (reconhecimento de fala)
TTS	*Text-to-Speech* (síntese de fala)
UI	*User Interface* (interface do usuário)
VAD	*Voice Activity Detection* (detecção de atividade de voz)
XDG	Especificações de área de trabalho do projeto freedesktop.org

SUMÁRIO

1 INTRODUÇÃO **12**
1.1 PROBLEMA DE PESQUISA 12
1.2 OBJETIVOS 13
1.2.1 Objetivo geral **13**
1.2.2 Objetivos específicos **13**
1.3 JUSTIFICATIVA 13
1.4 DELIMITAÇÃO DO ESCOPO 14
1.5 ORGANIZAÇÃO DO TRABALHO 14
2 FUNDAMENTAÇÃO TEÓRICA **15**
2.1 ASSISTENTES PESSOAIS POR VOZ 15
2.2 PROCESSAMENTO DE FALA 15
2.2.1 Detecção de atividade de voz **15**
2.2.2 Reconhecimento automático de fala **16**
2.2.3 Síntese de fala e prosódia **16**
2.3 MODELOS DE LINGUAGEM DE GRANDE PORTE E USO DE FERRAMENTAS 16
2.4 ARQUITETURA DE SOFTWARE 17
2.4.1 Arquitetura hexagonal **17**
2.4.2 Domínio, aplicação e raiz de composição **17**
2.4.3 Padrões de projeto empregados **17**
2.4.4 Funções de aptidão arquitetural **17**
2.5 PRINCÍPIOS SOLID 18
2.6 CÓDIGO LIMPO E REFATORAÇÃO 18
2.7 MÉTRICAS DE COMPLEXIDADE 19
2.8 TESTES AUTOMATIZADOS 19
2.9 QUALIDADE DE SOFTWARE 19
2.10 INTEGRIDADE DE DADOS E HISTÓRICOS AUDITÁVEIS 19
3 METODOLOGIA **21**
3.1 CARACTERIZAÇÃO DA PESQUISA 21
3.2 PROCESSO DE DESENVOLVIMENTO 21
3.3 DESENVOLVIMENTO ASSISTIDO POR AGENTE DE INTELIGÊNCIA ARTIFICIAL 22
3.4 AMBIENTE E FERRAMENTAS 22
3.5 CRITÉRIOS DE VALIDAÇÃO 23
3.6 NORMALIZAÇÃO DO DOCUMENTO 23
4 REQUISITOS DO SISTEMA **24**

4.1 PARTES INTERESSADAS E ATORES 24
4.2 REQUISITOS FUNCIONAIS 24
4.3 REQUISITOS NÃO FUNCIONAIS 28
4.4 REGRAS DE NEGÓCIO 31
5 ARQUITETURA E PROJETO 32
5.1 VISÃO GERAL 32
5.2 CAMADAS E MÓDULOS 32
5.3 CONTRATOS ENTRE AS CAMADAS 33
5.4 MODELO DE DADOS 34
5.5 MODELO DE CONCORRÊNCIA 35
5.6 FLUXOS DE EXECUÇÃO 36
5.6.1 Inicialização 36
5.6.2 Conversa em pipeline 36
5.6.3 Ativação por voz 37
5.6.4 Memória de longo prazo 37
5.6.5 Abertura de programas 37
5.6.6 Laço de ferramentas do modelo de linguagem 38
5.6.7 Respostas do modelo local 38
5.6.8 Animação da boca 38
5.7 DECISÕES DE PROJETO 39
6 CONSTRUÇÃO DO SISTEMA: PASSO A PASSO 41
6.1 INCREMENTO 1: PROTÓTIPO FUNCIONAL 41
6.2 INCREMENTO 2: BUSTO CENTRALIZADO E BOCA SUAVE 42
6.3 INCREMENTO 3: PERFIL DE VOZ 42
6.4 INCREMENTO 4: NITIDEZ, CAMISA FUTURISTA E BOCA NATURAL 43
6.5 INCREMENTO 5: ESTRUTURA PROFISSIONAL E TESTES (VERSÃO 1.0.0) . 43
6.6 INCREMENTO 6: REFATORAÇÃO COMPLETA (VERSÃO 2.0.0) 44
6.7 INCREMENTO 7: RENOMEAÇÃO PARA ROBOFAEL (VERSÃO 2.1.0) 44
6.8 INCREMENTO 8: GRAVAÇÃO DA VOZ REAL DO USUÁRIO 45
6.9 INCREMENTO 9: ATIVAÇÃO POR VOZ (VERSÃO 2.2.0) 45
6.10 INCREMENTO 10: FRASE “OK FAEL” E GRAMÁTICA RESTRITA (VERSÃO 2.3.0) 46
6.11 INCREMENTO 11: MEMÓRIA DE LONGO PRAZO E OUTRAS FUNÇÕES (VERSÃO 3.0.0) 46
6.12 INCREMENTO 12: GANHO AUTOMÁTICO (VERSÃO 3.0.1) 47
6.13 INCREMENTO 13: INTERNACIONALIZAÇÃO (VERSÃO 3.1.0) 47
6.14 INCREMENTO 14: INSTALADOR PARA UBUNTU (VERSÃO 3.2.0) 48
6.15 INCREMENTO 15: REVISÃO SOLID E CÓDIGO LIMPO (VERSÃO 3.3.0) . . . 48

6.16 INCREMENTO 16: DESEMPENHO E FLUIDEZ GUIADOS POR MEDIÇÃO (VERSÃO 3.4.0) . . . 49
6.17 INCREMENTO 17: NOME TÉCNICO ROBOFAEL (VERSÃO 3.5.0) . . . 50
6.18 INCREMENTO 18: MODELO DE LINGUAGEM LOCAL (VERSÃO 3.6.0) . . . 51
6.19 INCREMENTO 19: ESCOLHA DO MICROFONE E INICIALIZAÇÃO (VERSÃO 3.7.0) . . . 52
6.20 INCREMENTO 20: BANCO, BACKUP E MODELOS DE LINGUAGEM (VERSÃO 3.8.0) . . . 52
6.21 LIÇÕES APRENDIDAS NO PROCESSO . . . 53
7 REVISÃO SEGUNDO OS PRINCÍPIOS SOLID E O CÓDIGO LIMPO . . . 54
7.1 DIAGNÓSTICO . . . 54
7.2 RESPONSABILIDADE ÚNICA . . . 55
7.2.1 Divisão do orquestrador . . . 55
7.2.2 Sessão separada da composição e da interface . . . 55
7.3 ABERTO PARA EXTENSÃO, FECHADO PARA MODIFICAÇÃO . . . 55
7.4 SUBSTITUIÇÃO DE LISKOV . . . 56
7.5 SEGREGAÇÃO DE INTERFACES . . . 56
7.6 INVERSÃO DE DEPENDÊNCIA . . . 56
7.7 PRÁTICAS DE CÓDIGO LIMPO . . . 57
7.8 REGRAS PERMANENTES . . . 57
7.9 MÉTRICAS ANTES E DEPOIS . . . 58
8 TESTES E VALIDAÇÃO . . . 59
8.1 ESTRATÉGIA . . . 59
8.2 DUBLÊS DE TESTE . . . 59
8.3 SUÍTES DE TESTE . . . 60
8.4 TESTES DE ARQUITETURA . . . 61
8.5 ANÁLISE ESTÁTICA E INTEGRAÇÃO CONTÍNUA . . . 62
8.6 COBERTURA . . . 62
8.7 VALIDAÇÃO MANUAL . . . 63
8.8 ESTABILIDADE DA SUÍTE . . . 63
9 RESULTADOS E DISCUSSÃO . . . 65
9.1 ATENDIMENTO AOS REQUISITOS FUNCIONAIS . . . 65
9.2 ATENDIMENTO AOS REQUISITOS NÃO FUNCIONAIS . . . 65
9.3 DISCUSSÃO . . . 66
9.3.1 Latência percebida . . . 66
9.3.2 Privacidade e exatidão da ativação . . . 67
9.3.3 Memória auditável . . . 67
9.3.4 Princípios SOLID na prática . . . 67

9.3.5 Desenvolvimento assistido por inteligência artificial 68
9.4 LIMITAÇÕES 68
10 CONCLUSÃO 69
10.1 TRABALHOS FUTUROS 70
REFERÊNCIAS 71
GLOSSÁRIO 74
APÊNDICE A – RASTREABILIDADE ENTRE REQUISITOS E TESTES 75
APÊNDICE B – COMANDOS DE VOZ 77
APÊNDICE C – CONFIGURAÇÃO 78
APÊNDICE D – PROCEDIMENTOS DE INSTALAÇÃO, EXECUÇÃO E VERIFICAÇÃO 79

1 INTRODUÇÃO

Assistentes pessoais por voz tornaram-se parte do cotidiano em telefones celulares, caixas de som e automóveis. Em computadores pessoais, porém, sobretudo nos que utilizam o sistema operacional Linux, a oferta de assistentes integrados ao ambiente de trabalho ainda é limitada. As soluções comerciais costumam depender de serviços em nuvem permanentemente à escuta, oferecem pouca personalização da aparência e da voz e não guardam um histórico verificável das atividades realizadas pelo usuário. Ao mesmo tempo, o surgimento de modelos de linguagem de grande porte (*Large Language Models*, LLM), capazes de interpretar pedidos em linguagem natural e de acionar funções externas, reduziu drasticamente o esforço necessário para construir um assistente que compreenda frases livres, e não apenas comandos rígidos (Vaswani *et al.*, 2017).

Construir um assistente de voz útil, entretanto, envolve muito mais do que conectar um reconhecedor de fala a um modelo de linguagem. É preciso capturar o áudio com qualidade, decidir quando o usuário terminou de falar, sintetizar a resposta sem longas esperas, sincronizar a boca de um avatar com a voz, persistir dados de forma confiável e manter a interface gráfica fluida enquanto diversas tarefas concorrentes acontecem. Esse conjunto de preocupações, distribuído entre várias tecnologias, torna o software suscetível a acoplamento excessivo e à degradação de sua estrutura à medida que novas funções são acrescentadas, fenômeno descrito por Martin (2019) como o principal fator do custo crescente de manutenção.

Este trabalho descreve a concepção, a construção incremental e a validação do **Robofael**, um assistente pessoal de atividades diárias que roda localmente em um computador com Linux. O Robofael é representado por um avatar animado criado a partir de fotografias do próprio usuário, ouve e responde por voz, organiza tarefas e lembretes, responde perguntas sobre o que foi feito em determinado período, abre programas instalados e conversa em português ou em inglês. Além da descrição funcional, o trabalho dá ênfase à qualidade interna do software: a solução foi organizada segundo a arquitetura hexagonal (Cockburn, 2005) e, ao final, revisada de ponta a ponta à luz dos princípios SOLID (Martin, 2003) e das práticas de código limpo (Martin, 2009), com regras de projeto verificadas automaticamente por testes.

1.1 PROBLEMA DE PESQUISA

O problema que orienta este trabalho pode ser enunciado da seguinte forma: *como construir um assistente pessoal por voz para computadores com Linux que seja responsivo, preserve a privacidade do usuário, mantenha um histórico confiável de suas atividades e, ao mesmo tempo, conserve uma estrutura interna fácil de entender, testar e evoluir?*

1.2 OBJETIVOS

1.2.1 Objetivo geral

Desenvolver e validar um assistente pessoal de atividades diárias, com avatar animado e interação por voz, cuja arquitetura siga os princípios SOLID e as práticas de código limpo, com comportamento e regras de projeto verificados por testes automatizados.

1.2.2 Objetivos específicos

a) gerar, a partir de fotografias do usuário, um avatar em formato de busto com movimentação da boca sincronizada à voz sintetizada;
b) implementar o ciclo de conversa por voz (ouvir, compreender e responder) com latência percebida próxima à da primeira frase da resposta;
c) permitir a ativação sem as mãos por uma frase de chamada reconhecida localmente;
d) oferecer gestão de tarefas e lembretes e uma memória de longo prazo auditável, capaz de responder o que foi feito em um dia, em uma semana ou em outro período;
e) integrar o assistente ao ambiente Linux, permitindo abrir programas instalados por comando de voz;
f) suportar os idiomas português do Brasil e inglês dos Estados Unidos, com troca em tempo de execução;
g) disponibilizar um instalador padrão para o Ubuntu;
h) revisar a solução segundo os princípios SOLID e as práticas de código limpo, tornando as regras de arquitetura verificáveis por testes automatizados;
i) documentar os requisitos funcionais e não funcionais e todo o processo de construção.

1.3 JUSTIFICATIVA

A relevância do trabalho manifesta-se em três dimensões. Na dimensão do usuário, um assistente que roda no próprio computador, escuta passivamente apenas de forma local e guarda os dados em um arquivo sob controle do usuário atende a uma preocupação crescente com privacidade. A memória de longo prazo auditável, por sua vez, transforma o assistente em uma ferramenta de acompanhamento de produtividade: ao perguntar “o que eu fiz na semana passada?”, o usuário obtém uma resposta baseada em registros cuja integridade pode ser verificada.

Na dimensão técnica, o projeto reúne, em um único sistema de porte moderado, problemas clássicos de engenharia de software: concorrência entre captura de áudio, geração de texto, síntese de voz e animação; integração com serviços externos sujeitos a falhas; persistência transacional; e internacionalização. Isso o torna um estudo de caso adequado para avaliar como

princípios de projeto orientado a objetos se comportam em um contexto real.

Na dimensão acadêmica, o trabalho documenta um processo de desenvolvimento conduzido com o apoio de um agente de programação baseado em inteligência artificial e mostra que a qualidade estrutural, nesse cenário, pode ser preservada por meio de verificações automáticas, como testes de arquitetura, limites de complexidade e verificação estática de tipos, em vez de depender exclusivamente de disciplina manual.

1.4 DELIMITAÇÃO DO ESCOPO

O sistema destina-se a um único usuário por conta do sistema operacional e foi construído e testado no Ubuntu 24.04 com Python 3.12, embora o código seja compatível com Python 3.10 ou superior. O reconhecimento de fala para comandos usa o serviço de reconhecimento do Google acessado por meio da biblioteca SpeechRecognition, e a síntese de voz utiliza vozes neurais do serviço edge-tts, com o gTTS como reserva; ambos exigem conexão com a internet. A interpretação em linguagem natural usa a API do Claude ou, sem ela, um modelo local servido pelo Ollama, mais lento quando executado no processador; o modo offline cobre um conjunto definido de comandos. O treinamento de modelos acústicos próprios, a clonagem completa do timbre de voz e o suporte a outros sistemas operacionais estão fora do escopo deste trabalho.

1.5 ORGANIZAÇÃO DO TRABALHO

Além desta introdução, o trabalho está organizado em nove capítulos. O Capítulo 2 apresenta a fundamentação teórica sobre assistentes de voz, processamento de fala, modelos de linguagem, arquitetura de software, princípios SOLID e código limpo. O Capítulo 3 descreve a metodologia de desenvolvimento adotada. O Capítulo 4 especifica os requisitos funcionais e não funcionais. O Capítulo 5 detalha a arquitetura e os fluxos de execução. O Capítulo 6 relata, passo a passo, a construção incremental do sistema. O Capítulo 7 apresenta a revisão do código à luz dos princípios SOLID e do código limpo. O Capítulo 8 trata da estratégia de testes e da validação. O Capítulo 9 discute os resultados obtidos, e o Capítulo 10 apresenta as conclusões e os trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos que sustentam as decisões tomadas no projeto. Inicialmente são tratados os assistentes de voz e as tecnologias de fala; em seguida, os modelos de linguagem e o uso de ferramentas; por fim, os fundamentos de engenharia de software que orientaram a organização do código: arquitetura hexagonal, princípios SOLID, código limpo, métricas de complexidade, testes automatizados e qualidade de software.

2.1 ASSISTENTES PESSOAIS POR VOZ

Um assistente pessoal por voz é um sistema interativo que recebe pedidos falados, interpreta a intenção do usuário, executa ações e devolve uma resposta, também falada. Segundo Jurafsky; Martin (2009), sistemas de diálogo orientados a tarefas combinam tradicionalmente três componentes: o reconhecimento automático de fala, que converte o sinal acústico em texto; a compreensão da linguagem, que identifica a intenção e os parâmetros do pedido; e a geração e a síntese da resposta. Entre esses componentes, um gerenciador de diálogo decide quando ouvir, quando responder e como lidar com interrupções.

A qualidade percebida de um assistente depende fortemente da latência entre o fim da fala do usuário e o início da resposta. Nielsen (1993) associa tempos de resposta próximos de um segundo à manutenção do fluxo de pensamento do usuário; acima de dez segundos, a atenção se perde. Em um assistente de voz, a espera inclui a detecção do fim da fala, o reconhecimento, a geração do texto e a síntese do áudio, o que motiva arquiteturas que processam essas etapas de forma sobreposta, em vez de sequencial.

2.2 PROCESSAMENTO DE FALA

2.2.1 Detecção de atividade de voz

A detecção de atividade de voz (*Voice Activity Detection*, VAD) separa os trechos com fala dos trechos com silêncio ou ruído. Abordagens clássicas comparam a energia do sinal com um limiar estimado a partir do ruído de fundo, enquanto abordagens estatísticas modelam a distribuição do ruído e da fala (Sohn; Kim; Sung, 1999). No Robofael, a detecção é usada para decidir quando o usuário começou e terminou uma frase. Para isso, calibra-se o limiar a partir do ruído medido na abertura do microfone e exige-se uma pausa mínima para encerrar a captura. Ruídos curtos, como cliques, são descartados.

2.2.2 Reconhecimento automático de fala

O reconhecimento automático de fala (*Speech-to-Text*, STT) converte o áudio em texto. Sistemas modernos combinam modelos acústicos e de linguagem treinados em grandes volumes de dados (Jurafsky; Martin, 2009). O projeto emprega dois reconhecedores com papéis complementares. O primeiro é o Vosk, construído sobre o conjunto de ferramentas Kaldi (Povey *et al.*, 2011), que roda inteiramente no computador e permite restringir o vocabulário a uma gramática fechada (Alpha Cephei, 2026). O segundo é o serviço de reconhecimento do Google, mais preciso para frases livres, usado apenas depois que o usuário chama o assistente. A gramática restrita é especialmente útil para frases de ativação curtas, que um reconhecedor de vocabulário aberto tende a confundir com palavras mais frequentes.

2.2.3 Síntese de fala e prosódia

A síntese de fala (*Text-to-Speech*, TTS) produz áudio a partir de texto. Vozes neurais atuais permitem ajustar parâmetros prosódicos, como a frequência fundamental (o tom) e a velocidade de elocução (o ritmo). A frequência fundamental de uma voz pode ser estimada pela autocorrelação do sinal em janelas curtas, procurando o período que maximiza a semelhança do sinal consigo mesmo dentro da faixa típica da voz humana, entre 60 Hz e 400 Hz (Jurafsky; Martin, 2009). O Robofael usa essa estimativa para comparar a voz gravada do usuário com a voz sintética e calcular o deslocamento de tom e de ritmo que as aproxima.

2.3 MODELOS DE LINGUAGEM DE GRANDE PORTE E USO DE FERRAMENTAS

Os modelos de linguagem de grande porte baseiam-se na arquitetura *Transformer*, cujo mecanismo de atenção permite modelar dependências de longo alcance no texto (Vaswani *et al.*, 2017). Além de gerar texto, esses modelos podem ser instruídos a solicitar a execução de funções externas, as chamadas ferramentas (*tools*): o modelo devolve o nome da ferramenta e os argumentos em formato estruturado; a aplicação executa a função e devolve o resultado; e o modelo continua a resposta. A API do Claude oferece esse mecanismo com transmissão em fluxo contínuo (*streaming*) do texto e das entradas das ferramentas (Anthropic, 2026). Quando as entradas são transmitidas antes de estarem completas, cabe à aplicação validá-las contra o esquema declarado antes de executar qualquer ação, porque uma entrada truncada não pode ser tratada como válida.

Modelos menores, com alguns bilhões de parâmetros, podem ser executados no próprio computador por servidores locais como o Ollama, que expõe uma API HTTP de conversa com suporte a ferramentas (Ollama, 2026). A execução local dispensa a conexão e não envia o texto a terceiros, mas, sem placa de vídeo, as respostas são mais lentas, e modelos pequenos tendem a escolher ferramentas com menos precisão. Neste trabalho, essa diferença foi tratada restringindo as ferramentas oferecidas ao modelo local (Capítulo 6).

2.4 ARQUITETURA DE SOFTWARE

2.4.1 Arquitetura hexagonal

A arquitetura hexagonal, também chamada de arquitetura de portas e adaptadores, foi proposta por Cockburn (2005) com o objetivo de isolar a lógica da aplicação dos mecanismos de entrada e saída. A aplicação define *portas*, isto é, interfaces que descrevem o que ela precisa do mundo externo ou o que oferece a ele, e cada tecnologia concreta é conectada por meio de um *adaptador* que implementa a porta. Com isso, banco de dados, interface gráfica, serviços de voz e modelos de linguagem tornam-se substituíveis, e a lógica central pode ser testada com dublês simples. Martin (2019) generaliza a ideia na chamada regra de dependência: as dependências de código-fonte devem apontar sempre para dentro, das camadas de detalhe para as camadas de política.

2.4.2 Domínio, aplicação e raiz de composição

Evans (2004) propõe que as regras de negócio sejam expressas em um modelo de domínio independente de infraestrutura, com entidades, objetos de valor e serviços de domínio. Os casos de uso que coordenam esse modelo formam a camada de aplicação. A ligação entre abstrações e implementações concretas é feita por injeção de dependências (Fowler, 2004), idealmente concentrada em um único ponto do programa, a raiz de composição, que monta o grafo de objetos na inicialização.

2.4.3 Padrões de projeto empregados

Diversos padrões catalogados por Gamma *et al.* (1994) aparecem na solução: o *Observer*, na forma de um barramento de eventos que desacopla produtores e consumidores; o *State*, em uma máquina de estados explícita que controla o ciclo de conversa; o *Strategy* e a *Chain of Responsibility*, nas cadeias de interpretadores de intenções e de períodos; e o *Adapter*, nos componentes que encapsulam bibliotecas externas.

2.4.4 Funções de aptidão arquitetural

Ford; Parsons; Kua (2017) propõem tratar características arquiteturais desejadas como funções de aptidão (*fitness functions*): verificações objetivas e, preferencialmente, automatizadas, executadas continuamente para indicar se a arquitetura ainda atende ao que se espera dela. Um exemplo é um teste que inspeciona as dependências entre módulos e falha quando uma camada interna passa a depender de uma externa. Essa ideia fundamenta os testes de arquitetura descritos no Capítulo 7.

2.5 PRINCÍPIOS SOLID

O acrônimo SOLID reúne cinco princípios de projeto orientado a objetos sistematizados por Martin (2003):

a) **princípio da responsabilidade única** (*Single Responsibility Principle*, SRP): um módulo deve ter um, e apenas um, motivo para mudar; em outras palavras, deve responder a um único ator ou preocupação (Martin, 2019);
b) **princípio aberto/fechado** (*Open-Closed Principle*, OCP): entidades de software devem estar abertas para extensão e fechadas para modificação, formulação atribuída a Meyer (1988); acrescentar um comportamento não deve exigir a alteração de código que já funciona;
c) **princípio da substituição de Liskov** (*Liskov Substitution Principle*, LSP): objetos de um subtipo devem poder substituir objetos do tipo base sem alterar as propriedades desejáveis do programa (Liskov; Wing, 1994);
d) **princípio da segregação de interfaces** (*Interface Segregation Principle*, ISP): clientes não devem ser forçados a depender de métodos que não usam, o que favorece interfaces pequenas e específicas;
e) **princípio da inversão de dependência** (*Dependency Inversion Principle*, DIP): módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações.

Na linguagem Python, as abstrações exigidas pelo DIP e pelo ISP podem ser expressas por protocolos (`typing.Protocol`), que definem contratos verificados estruturalmente pelo verificador de tipos, sem exigir herança explícita (Levkivskyi; Lehtosalo; Langa, 2017). Essa característica é explorada em todo o projeto.

2.6 CÓDIGO LIMPO E REFATORAÇÃO

Martin (2009) caracteriza o código limpo como aquele que é fácil de ler e de modificar por outras pessoas: nomes que revelam a intenção, funções pequenas que fazem uma única coisa, poucos argumentos, ausência de duplicação e de efeitos colaterais ocultos, e tratamento de erros que não obscurece a lógica. O princípio da não repetição (*Don't Repeat Yourself*) de Hunt; Thomas (1999) complementa essas práticas. A refatoração, segundo Fowler (2018), é a alteração da estrutura interna do software sem mudar seu comportamento observável, feita em pequenos passos e protegida por testes automatizados. Entre as técnicas aplicadas neste trabalho estão a extração de classe, a extração de função, a substituição de condicionais por polimorfismo ou por tabelas de despacho e a introdução de objetos-parâmetro.

2.7 MÉTRICAS DE COMPLEXIDADE

A complexidade ciclomática, proposta por McCabe (1976), mede o número de caminhos linearmente independentes no grafo de controle de uma função. Ela corresponde, de forma aproximada, ao número de decisões mais um. Valores altos indicam funções difíceis de entender e de testar por completo, e o limite de 10 sugerido pelo próprio autor é amplamente adotado como teto prático. Neste trabalho, a métrica foi usada tanto para localizar os pontos que precisavam de refatoração quanto como regra permanente do analisador estático.

2.8 TESTES AUTOMATIZADOS

O desenvolvimento orientado a testes, descrito por Beck (2003), popularizou a prática de escrever testes automatizados pequenos e rápidos que documentam o comportamento esperado. Cohn (2009) propõe a pirâmide de testes: muitos testes unitários na base, uma quantidade menor de testes de integração e poucos testes de ponta a ponta, que são mais lentos e frágeis. Meszaros (2007) cataloga os dublês de teste, como *stubs*, *fakes* e *spies*, que substituem colaboradores reais para isolar o comportamento sob teste. A combinação de arquitetura hexagonal com dublês permite testar a lógica de conversa sem microfone, sem alto-falante e sem rede.

2.9 QUALIDADE DE SOFTWARE

A norma ISO/IEC 25010 define um modelo de qualidade de produto com características como adequação funcional, eficiência de desempenho, compatibilidade, usabilidade, confiabilidade, segurança, manutenibilidade e portabilidade (International Organization for Standardization, 2011). Esse modelo foi usado como referência para organizar os requisitos não funcionais do Capítulo 4. Na engenharia de requisitos, Sommerville (2018) e Pressman; Maxim (2016) distinguem os requisitos funcionais, que descrevem o que o sistema deve fazer, dos requisitos não funcionais, que restringem como ele o faz, distinção também adotada neste trabalho.

2.10 INTEGRIDADE DE DADOS E HISTÓRICOS AUDITÁVEIS

Um histórico é auditável quando permite reconstruir o que aconteceu e detectar alterações indevidas. Duas técnicas complementares sustentam essa propriedade no projeto. A primeira é o armazenamento somente de acréscimo: os registros nunca são alterados nem removidos, e restrições do próprio banco de dados recusam essas operações. A segunda é o encadeamento de resumos criptográficos: cada registro guarda o resumo SHA-256 (National Institute of Standards and Technology, 2015) do anterior, de modo que qualquer modificação, remoção ou inserção indevida quebra a cadeia a partir do ponto alterado. O banco de dados SQLite oferece transações atômicas e gatilhos (*triggers*) adequados para implementar essas

garantias em um único arquivo local (SQLite, 2026).

3 METODOLOGIA

Este capítulo descreve o tipo de pesquisa, o processo de desenvolvimento, o ambiente e as ferramentas utilizadas e os critérios adotados para validar o sistema.

3.1 CARACTERIZAÇÃO DA PESQUISA

Quanto à natureza, trata-se de pesquisa aplicada, voltada à solução de um problema concreto por meio da construção de um artefato de software. Quanto aos procedimentos, caracteriza-se como um estudo de caso de desenvolvimento, no qual o próprio processo de construção, as decisões de projeto e os problemas encontrados constituem objeto de análise. A abordagem é predominantemente qualitativa, complementada por medidas quantitativas de qualidade interna, como o número de testes, a cobertura de código, a complexidade ciclomática e o tamanho dos módulos, e de desempenho, como os tempos de inicialização, de resposta e de interrupção.

3.2 PROCESSO DE DESENVOLVIMENTO

Adotou-se um processo iterativo e incremental (Larman; Basili, 2003). Cada incremento partiu de um pedido do usuário, que atuou simultaneamente como cliente e como responsável pelas decisões de produto, e terminou com o software executável, testado e documentado. Ao todo, foram realizados vinte incrementos, descritos no Capítulo 6. Cada um deles seguiu as etapas abaixo:

a) **entendimento do pedido**: identificação do objetivo e das restrições; quando havia mais de uma solução razoável com consequências diferentes para o usuário, a escolha era submetida a ele, como na decisão entre o reconhecimento local e o remoto da frase de ativação ou no ajuste apenas do tom da voz;
b) **investigação**: leitura do código afetado, medições, como volume do microfone, transcrições reais e tempos de execução, e experimentos curtos para confirmar hipóteses;
c) **implementação**: alterações pequenas e reversíveis, preservando a arquitetura existente;
d) **verificação**: execução da suíte de testes, da análise estática de estilo e de complexidade (ruff) e da verificação de tipos (mypy); acréscimo de testes para todo comportamento novo ou defeito corrigido;
e) **validação com o usuário**: execução real do assistente, com análise de registros detalhados (opção `-v`) para calibrar parâmetros como limiares de detecção e ganho do microfone;
f) **documentação**: atualização do arquivo README, do registro de mudanças

(CHANGELOG) e do número de versão segundo o versionamento semântico (Preston-Werner, 2013).

3.3 DESENVOLVIMENTO ASSISTIDO POR AGENTE DE INTELIGÊNCIA ARTIFICIAL

A implementação foi realizada com o apoio do Claude Code, um agente de programação baseado em modelo de linguagem que lê e edita arquivos, executa comandos e testes e interage com o usuário em linguagem natural. O usuário definiu os objetivos e as prioridades, tomou as decisões de produto e validou o comportamento em uso real; o agente propôs e implementou as soluções, executou as verificações e relatou os resultados. Esse arranjo impôs uma disciplina adicional: como uma alteração pode ser produzida rapidamente, a garantia de que ela não degrada o sistema precisa vir de verificações automáticas. Por isso, testes, limites de complexidade, verificação de tipos e testes de arquitetura foram tratados como parte do produto, e não como atividades opcionais.

3.4 AMBIENTE E FERRAMENTAS

O Quadro 1 relaciona as tecnologias utilizadas e a finalidade de cada uma.

Quadro 1 – Tecnologias e ferramentas utilizadas

Tecnologia	**Finalidade no projeto**
Python 3.12 (compatível com 3.10 ou superior)	Linguagem de implementação
Tkinter	Interface gráfica (janela, palco do avatar, painel e configurações)
Pillow e NumPy (Harris *et al.*, 2020)	Recorte, nitidez, troca de roupa e geração dos quadros do avatar; análise de áudio
SpeechRecognition e PyAudio	Captura do microfone e reconhecimento de fala pelo serviço do Google
Vosk (modelo `vosk-model-small-pt-0.3`)	Reconhecimento local da frase de ativação, com gramática restrita
edge-tts e gTTS	Síntese de voz neural com ajuste de tom e ritmo; voz de reserva
pygame	Decodificação e reprodução do áudio em memória
SDK oficial da Anthropic (Claude, modelo `claude-opus-5-5`)	Compreensão de linguagem natural com ferramentas e transmissão em fluxo contínuo
SQLite	Agenda, histórico auditável e preferências
pytest e pytest-cov	Testes automatizados e medição de cobertura
ruff	Análise estática de estilo, de defeitos comuns e de complexidade
mypy	Verificação estática de tipos
pre-commit e GitHub Actions com Xvfb	Verificações antes de cada versão e integração contínua, inclusive da interface gráfica
dpkg-deb e fakeroot	Construção do pacote de instalação para Ubuntu
GNU Make	Automação das tarefas de desenvolvimento

Fonte: elaboração própria (2026).

O ambiente de desenvolvimento e de testes foi um computador com Ubuntu 24.04 (núcleo Linux 6.8), oito núcleos de processamento, microfone interno e um microfone USB, com Python 3.12 em ambiente virtual isolado.

3.5 CRITÉRIOS DE VALIDAÇÃO

O sistema foi considerado válido quando atendeu, simultaneamente, aos critérios a seguir:

a) todos os testes automatizados aprovados, inclusive os de interface gráfica e os de arquitetura;
b) nenhuma ocorrência apontada pela análise estática de estilo e de complexidade, com complexidade ciclomática máxima de 10 por função, nem pela verificação de tipos;
c) cada requisito funcional associado a pelo menos um teste automatizado ou a um procedimento de verificação manual documentado (Apêndice A);
d) metas de desempenho atingidas, conforme os requisitos não funcionais;
e) instalador testado em ambiente isolado, sem alterar o sistema;
f) execução real do assistente pelo usuário, com as funções principais operando.

Para a revisão segundo os princípios SOLID, adotou-se o critério de preservação do comportamento: a suíte completa deveria permanecer aprovada após cada passo de refatoração e, quando a alteração afetasse a geração de imagens, a saída deveria permanecer idêntica pixel a pixel.

3.6 NORMALIZAÇÃO DO DOCUMENTO

Este documento segue as normas da Associação Brasileira de Normas Técnicas para trabalhos acadêmicos (Associação Brasileira de Normas Técnicas, 2011), referências (Associação Brasileira de Normas Técnicas, 2018) e citações (Associação Brasileira de Normas Técnicas, 2023). Por se tratar de uma documentação exclusivamente textual, os fluxos e as estruturas do sistema são descritos em prosa, em listas numeradas e em quadros, sem ilustrações gráficas.

4 REQUISITOS DO SISTEMA

Este capítulo especifica os requisitos do Robofael. Seguindo Sommerville (2018), os requisitos funcionais (RF) descrevem os serviços que o sistema oferece, e os requisitos não funcionais (RNF) descrevem restrições e atributos de qualidade, organizados segundo as características da ISO/IEC 25010 (International Organization for Standardization, 2011). Cada requisito recebeu um identificador, uma prioridade e a indicação de como é verificado. A prioridade segue três níveis: **essencial**, sem o qual o sistema não cumpre seu propósito; **importante**, que afeta significativamente a experiência; e **desejável**, que agrega valor sem comprometer o uso básico. A rastreabilidade entre os requisitos e os testes automatizados é apresentada no Apêndice A.

4.1 PARTES INTERESSADAS E ATORES

O sistema possui um único ator humano, o **usuário**, que é a pessoa dona do computador e das fotografias que dão origem ao avatar. Ele interage por voz, teclado e mouse. Há ainda três atores externos: o **serviço de modelo de linguagem** (Claude, na nuvem, ou um modelo local servido pelo Ollama), que interpreta pedidos em linguagem natural; os **serviços de fala** (reconhecimento do Google e síntese do edge-tts e do gTTS), acessados pela internet; e o **sistema operacional**, que fornece o microfone, o áudio, a lista de programas instalados e os diretórios de dados do usuário.

4.2 REQUISITOS FUNCIONAIS

Os requisitos funcionais foram agrupados por área: interação e avatar (Quadro 2), agenda e memória (Quadro 3), inteligência e integração com o sistema (Quadro 4) e configuração, idioma e distribuição (Quadro 5). Os requisitos incorporados nas versões 3.6.0 a 3.8.0 estão reunidos no Quadro 6.

Quadro 2 – Requisitos funcionais de interação e avatar

ID	Descrição	Prioridade	Verificação
RF01	Exibir um avatar em formato de busto, centralizado, gerado a partir das fotografias do usuário, com roupa substituída por uma camisa de estilo futurista e nitidez realçada.	Essencial	Testes de avatar
RF02	Animar a boca do avatar em sincronia com a voz sintetizada, variando abertura (12 níveis) e formato (três formatos de vogal), com transição suave e leve balanço em repouso.	Essencial	Testes de animação

ID	Descrição	Prioridade	Verificação
RF03	Iniciar a escuta de um comando pela tecla ESPAÇO ou pelo botão de microfone, exibindo o nível real da voz captada.	Essencial	Testes do orquestrador; uso real
RF04	Ativar o assistente sem as mãos ao ouvir a frase “ok Fael”, confirmando com um sinal sonoro e passando a ouvir o comando.	Essencial	Testes de ativação
RF05	Aceitar a frase de ativação e o comando em uma única fala (por exemplo, “ok Fael, anota tarefa comprar pão”), extraindo apenas o comando.	Importante	Testes de ativação (transcrições reais)
RF06	Responder por voz neural, com tom e ritmo ajustáveis, inclusive a partir do perfil gravado da voz do usuário.	Essencial	Testes de voz e configurações
RF07	Aceitar comandos digitados em uma caixa de texto, com o mesmo tratamento dos comandos falados; um texto enviado com o assistente ocupado interrompe o que ele faz e é atendido em seguida.	Importante	Testes de interface e do orquestrador
RF08	Exibir legendas com a fala reconhecida do usuário e com a resposta à medida que ela é gerada.	Importante	Testes de interface
RF09	Permitir interromper, a qualquer momento, a escuta, a geração ou a fala da resposta (ESPAÇO, ESC ou botão contextual).	Essencial	Testes do orquestrador
RF10	Indicar visualmente o estado do assistente (iniciando, parado, ouvindo, pensando, falando), com rótulo do botão adequado a cada estado.	Importante	Testes de estados e de interface
RF11	Saudar o usuário ao iniciar, informando o número de tarefas pendentes, o modo offline (se for o caso) e como chamar o assistente.	Desejável	Testes de sessão e de idiomas
RF12	Exibir avisos na tela para microfone ausente, voz captada baixa demais, falhas de áudio e de avatar e desligamento da escuta passiva.	Importante	Testes de ganho, ativação e interface
RF13	Oferecer modo contínuo, em que o assistente volta a ouvir após cada resposta até que o usuário se despeça.	Desejável	Testes do orquestrador
RF14	Trocar o avatar entre as fotografias disponíveis (tecla T ou configurações), sem travar a interface.	Desejável	Testes de configurações

Fonte: elaboração própria (2026).

Quadro 3 – Requisitos funcionais de agenda e memória

ID	Descrição	Prioridade	Verificação
RF15	Criar tarefas por voz ou por texto.	Essencial	Testes da agenda e do modo offline
RF16	Listar as tarefas pendentes e, opcionalmente, as concluídas.	Essencial	Testes da agenda e de ferramentas

ID	Descrição	Prioridade	Verificação
RF17	Concluir, reabrir e remover tarefas por voz ou pelo painel (duplo clique e tecla Delete).	Essencial	Testes da agenda e de interface
RF18	Criar lembretes para uma data e hora ou para daqui a N minutos e anunciá-los por voz no horário.	Essencial	Testes do agendador
RF19	Listar e remover lembretes futuros.	Importante	Testes da agenda
RF20	Registrar automaticamente, em histórico, cada criação, conclusão, reabertura e remoção de tarefa e cada criação, aviso e remoção de lembrete, com data e hora.	Essencial	Testes do SQLite
RF21	Responder “o que eu fiz” em um dia, semana ou período (ontem, anteontem, um dia da semana, uma data, semana passada, esta semana, últimos N dias, mês passado), considerando apenas o resultado líquido (tarefa reaberta não conta como feita).	Essencial	Testes de memória
RF22	Verificar a integridade do histórico e apontar o primeiro registro alterado, removido ou inserido indevidamente (comando `make integridade`).	Importante	Testes do SQLite
RF23	Migrar automaticamente a agenda do formato JSON das versões anteriores para o banco de dados, preservando o arquivo original renomeado.	Importante	Testes do SQLite

Fonte: elaboração própria (2026).

Quadro 4 – Requisitos funcionais de inteligência e integração com o sistema

ID	Descrição	Prioridade	Verificação
RF24	Interpretar pedidos em linguagem natural com um modelo de linguagem que opera a agenda, o histórico e os programas por meio de ferramentas.	Essencial	Testes do modelo Claude e de ferramentas
RF25	Funcionar em modo offline quando o modelo de linguagem estiver indisponível, atendendo a comandos de tarefas, lembretes relativos, hora, data, histórico e programas.	Essencial	Testes do modo offline
RF26	Iniciar uma nova conversa quando solicitado (“nova conversa”), sem perder a agenda.	Desejável	Testes do modelo Claude
RF27	Informar a hora e a data atuais.	Importante	Testes do modo offline
RF28	Abrir programas instalados no Linux pelo nome ou por sinônimos (por exemplo, “navegador”, “calculadora”), usando o navegador padrão quando pedido e solicitando escolha em caso de ambiguidade.	Importante	Testes de programas
RF29	Listar os programas instalados, com filtro opcional pelo nome.	Desejável	Testes de programas

Fonte: elaboração própria (2026).

Quadro 5 – Requisitos funcionais de configuração, idioma e distribuição

ID	Descrição	Prioridade	Verificação
RF30	Gravar amostras da voz do usuário e gerar um perfil com o deslocamento de tom e de ritmo que aproxima a voz sintética da voz dele.	Importante	Testes de perfil de voz
RF31	Oferecer tela de configurações com idioma, avatar, voz base, tom, ritmo, botão para ouvir o resultado, uso do perfil gravado e ativação por voz, com opções de salvar e de cancelar (que desfaz o teste).	Importante	Testes de configurações
RF32	Guardar as preferências entre sessões, com precedência sobre o perfil gravado e sobre os valores padrão.	Importante	Testes de configurações
RF33	Responder em português do Brasil ou em inglês dos Estados Unidos, trocando ao vivo a interface, as respostas, a voz, o reconhecimento de fala e as instruções do modelo.	Importante	Testes de idiomas
RF34	Entender comandos nos dois idiomas, independentemente do idioma das respostas.	Desejável	Testes de idiomas
RF35	Aceitar opções de linha de comando (avatar, modo contínuo, desligar ativação, idioma e registros detalhados) e configuração por variáveis de ambiente.	Desejável	Testes de configuração
RF36	Ser instalável no Ubuntu por um pacote `.deb`, com comandos no terminal, atalhos e ícone no menu de aplicativos e remoção que preserva os dados do usuário.	Importante	Teste do pacote em ambiente isolado

Fonte: elaboração própria (2026).

Quadro 6 – Requisitos funcionais incorporados nas versões 3.6.0 a 3.8.0

ID	Descrição	Prioridade	Verificação
RF37	Sem o Claude, conversar e atender pedidos de agenda em linguagem livre com um modelo local (Ollama), usando apenas ferramentas que criam ou consultam, e voltar ao modo offline se o modelo local deixar de responder.	Importante	Testes do modelo local
RF38	Exibir nas configurações o modelo de linguagem em uso e os modelos compatíveis do Claude e do Ollama, informando o motivo quando um provedor não puder ser consultado.	Desejável	Testes de banco e modelos; testes de configurações
RF39	Escolher o microfone entre os conectados ao computador, com a troca valendo na escuta seguinte, inclusive para a escuta passiva e a gravação da voz.	Importante	Testes de microfone e inicialização

ID	Descrição	Prioridade	Verificação
RF40	Oferecer, como opções, abrir o assistente ao entrar no computador e pelo comando `robofael` em qualquer terminal.	Desejável	Testes de microfone e inicialização
RF41	Mudar o banco de dados para outra pasta com o assistente em uso, sem apagar o arquivo anterior e sem sobrescrever um banco existente, mantendo a escolha nas aberturas seguintes.	Importante	Testes de banco e modelos
RF42	Fazer backup do banco de dados em qualquer local, com a cópia conferida antes de ser aceita.	Importante	Testes de banco e modelos

Fonte: elaboração própria (2026).

4.3 REQUISITOS NÃO FUNCIONAIS

Os requisitos não funcionais estão organizados em duas partes: desempenho, confiabilidade e segurança (Quadro 7); e usabilidade, manutenibilidade e portabilidade (Quadro 8). Sempre que possível, foram definidos com metas mensuráveis.

Quadro 7 – Requisitos não funcionais de desempenho, confiabilidade e segurança

ID	Característica	Descrição e meta	Verificação
RNF01	Desempenho (tempo)	A janela deve ficar visível em menos de 1 s após a abertura; o trabalho lento (modelo, microfone, áudio e avatar) ocorre em segundo plano.	Medição; teste de inicialização
RNF02	Desempenho (tempo)	A primeira palavra da resposta deve ser falada assim que a primeira frase for gerada e sintetizada, sem esperar o restante da resposta.	Teste que prova o pipeline
RNF03	Desempenho (tempo)	Uma interrupção deve surtir efeito em cerca de 10 ms em qualquer estágio (ouvindo, pensando ou falando).	Testes do orquestrador
RNF04	Desempenho (tempo)	Lembretes devem ser anunciados com atraso inferior a 0,1 s em relação ao horário marcado.	Testes do agendador
RNF05	Desempenho (recursos)	A animação deve rodar a 60 quadros por segundo com o assistente ativo e a 30 em repouso, com compensação de deriva, reenviando ao Tk apenas o que mudou e trocando apenas imagens pré-renderizadas; falas repetidas e quadros do avatar devem vir de cache em disco (LRU de 300 falas; cache do avatar invalidado pelo resumo do código).	Testes de animação e de voz

ID	Característica	Descrição e meta	Verificação
RNF06	Confiabilidade (integridade)	Toda alteração da agenda e seus registros de histórico devem ser gravados na mesma transação; o histórico é somente de acréscimo e encadeado por SHA-256.	Testes do SQLite
RNF07	Confiabilidade (consistência)	O histórico da conversa com o modelo só cresce por acréscimo; uma rodada que falhe, seja recusada ou interrompida é descartada por inteiro.	Testes do modelo Claude
RNF08	Confiabilidade (tolerância a falhas)	Na ausência do modelo, usar o modo offline; na falha da voz neural, usar a voz de reserva; sem microfone, aceitar texto; após três falhas seguidas da escuta passiva, desligá-la com aviso.	Testes de composição, ativação e voz
RNF09	Confiabilidade (concorrência)	Apenas uma interação por vez; somente a thread principal acessa a interface gráfica; cada interação possui um token de cancelamento próprio, criado de forma síncrona.	Testes do orquestrador e do barramento
RNF10	Segurança (privacidade)	A escuta passiva deve ser processada localmente; o áudio só é enviado a serviços externos depois da ativação ou de um comando explícito. Os dados ficam em arquivos locais do usuário.	Revisão de código; testes de ativação
RNF11	Segurança (integridade de entradas)	As entradas das ferramentas do modelo devem ser validadas contra o esquema (tipos, campos obrigatórios e campos extras) antes da execução; entradas truncadas nunca são executadas; programas são abertos sem interpretador de comandos.	Testes de ferramentas e de programas
RNF12	Segurança (segredos)	Credenciais vêm de variáveis de ambiente e não são registradas em logs nem gravadas em arquivos do projeto.	Revisão de código
RNF13	Exatidão (ativação)	A frase de ativação não deve disparar com frases parecidas: meta de zero falsos positivos no conjunto de calibração, inclusive para o nome do usuário (“ok Rafael”).	Testes de ativação com transcrições reais

Fonte: elaboração própria (2026).

Quadro 8 – Requisitos não funcionais de usabilidade, manutenibilidade e portabilidade

ID	Característica	Descrição e meta	Verificação
RNF14	Usabilidade (operabilidade)	Todas as ações principais devem ter atalho de teclado (ESPAÇO, ENTER, ESC, T, Ctrl+Q); o botão principal muda de rótulo conforme o estado.	Testes de interface

ID	Característica	Descrição e meta	Verificação
RNF15	Usabilidade (estética e estabilidade)	A janela tem tamanho fixo, para que o palco desenhado em coordenadas absolutas não se desloque ao maximizar.	Teste de tamanho fixo
RNF16	Usabilidade (retorno)	O nível do microfone, o estado atual, as legendas e os avisos devem ser exibidos continuamente.	Testes de interface
RNF17	Usabilidade (inclusão)	As instruções ao modelo e as mensagens não devem atribuir gênero ao usuário.	Revisão do catálogo de mensagens
RNF18	Internacionalização	Todas as mensagens ficam em um catálogo único, com as duas línguas lado a lado; nenhuma tradução pode faltar e os campos de formatação devem coincidir.	Testes de idiomas
RNF19	Manutenibilidade (modularidade)	Arquitetura hexagonal com regra de dependência verificada por teste; domínio e portas sem bibliotecas de infraestrutura.	Testes de arquitetura
RNF20	Manutenibilidade (analisabilidade)	Complexidade ciclomática de no máximo 10 por função; funções curtas; nomes padronizados; nenhum parâmetro sem uso; tipos verificados estaticamente.	ruff e mypy na integração contínua
RNF21	Manutenibilidade (testabilidade)	Cobertura mínima de 80% das linhas e ramificações; dublês para todos os serviços externos; testes de interface com servidor gráfico virtual.	pytest-cov; integração contínua
RNF22	Manutenibilidade (modificabilidade)	Acrescentar uma intenção offline, uma ferramenta do modelo ou uma expressão de período não deve exigir alteração do código de despacho.	Testes de arquitetura
RNF23	Portabilidade (adaptabilidade)	Compatível com Python 3.10 ou superior; caminhos de dados configuráveis e, quando instalado, nos diretórios XDG do usuário.	Integração contínua (3.10 e 3.12)
RNF24	Portabilidade (instalabilidade)	Instalação sem internet, com as dependências Python incluídas no pacote; o ambiente Python é criado na instalação e verificado com `pip check`.	Teste do pacote
RNF25	Rastreabilidade de mudanças	Versões numeradas segundo o versionamento semântico, com registro de mudanças por versão.	Revisão do CHANGELOG
RNF26	Desempenho (recursos)	Em repouso, o assistente deve consumir no máximo 10% de um núcleo de processamento; um quadro em repouso não pode reenviar ao Tk o que não mudou.	Medição; `test_renderizacao`
RNF27	Desempenho (fluidez)	Nenhum trabalho da thread da interface pode ultrapassar o orçamento de um quadro; conversões de imagem são feitas aos poucos. Entre frases faladas, a pausa não deve exceder 0,2 s.	`test_renderizacao`; testes de áudio

ID	Característica	Descrição e meta	Verificação
RNF28	Confiabilidade (preservação dos dados)	Nenhuma operação sobre o banco pode apagar ou sobrescrever dados: cópias são conferidas antes do uso, o arquivo anterior é renomeado, e um banco escolhido inacessível não é substituído por um banco vazio. Os testes não podem alterar a pasta pessoal real.	Testes de banco e modelos; fixture de pasta pessoal isolada

Fonte: elaboração própria (2026).

4.4 REGRAS DE NEGÓCIO

Além dos requisitos, algumas regras de negócio orientam o comportamento do sistema:

RN1: comandos de agenda têm prioridade sobre a abertura de programas: “anota tarefa abrir conta no banco” cria uma tarefa, e não tenta abrir um programa chamado “conta”;

RN2: períodos ditos pelo usuário referem-se sempre ao passado: “segunda-feira” é a mais recente antes de hoje, e “dia 30”, dito em um dia 22, é o dia 30 do mês anterior;

RN3: datas numéricas seguem o idioma das respostas: 05/10 é 5 de outubro em português e 10 de maio em inglês;

RN4: para responder “o que eu fiz”, vale o estado líquido no período: uma tarefa concluída e depois reaberta não é contada como feita;

RN5: a precedência das configurações é: preferência salva, depois opção de linha de comando ou variável de ambiente e, por fim, valor padrão; no caso da voz, o perfil gravado vem antes do padrão;

RN6: ao trocar de idioma, a voz é substituída pela voz padrão do novo idioma caso a atual não o fale, e a conversa com o modelo recomeça;

RN7: nenhum dado do usuário é apagado automaticamente: arquivos corrompidos, migrados ou substituídos por uma mudança de local do banco são renomeados e preservados, e a remoção do pacote mantém a pasta de dados;

RN8: com o modelo local, ações que alteram ou removem dados e a abertura de programas só acontecem por comandos explícitos, nunca por escolha do modelo.

5 ARQUITETURA E PROJETO

Este capítulo descreve a organização do Robofael: as camadas e seus módulos, os contratos entre elas, o modelo de dados, o modelo de concorrência e os principais fluxos de execução. Por se tratar de uma documentação textual, os fluxos são apresentados como sequências numeradas de passos.

5.1 VISÃO GERAL

O Robofael é uma aplicação de área de trabalho escrita em Python, distribuída como o pacote `robofael` e executada como um único processo. A arquitetura segue o modelo hexagonal (Cockburn, 2005): no centro estão o domínio e os casos de uso; nas bordas, os adaptadores para cada tecnologia; e um único ponto, a raiz de composição, conhece as implementações concretas e monta o sistema. A regra de dependência (Martin, 2019) é respeitada em todo o código e verificada por testes automatizados (Seção 8.4).

5.2 CAMADAS E MÓDULOS

O Quadro 9 descreve as camadas, os módulos que as compõem e as dependências permitidas a cada uma.

Quadro 9 – Camadas da arquitetura, módulos e dependências permitidas

Camada	**Módulos e responsabilidade**	**Pode depender de**
Domínio (`dominio`)	Entidades e regras puras: `modelos` (tarefa, lembrete, registro de auditoria), `agenda` (serviço de domínio transacional), `repositorios` (contrato de persistência exigido pela agenda), `estados` (máquina de estados), `eventos`, `linguagem` (despedidas, pedidos de nova conversa, perguntas sobre o histórico), `periodos` (interpretação de períodos), `programas` (busca do programa pedido) e `ativacao` (gramática e busca fonética da frase de ativação).	Somente `i18n`
Portas (`portas`)	Contratos (`Protocol`) pequenos: modelo de linguagem, reconhecedor de fala, captura de áudio, detector de ativação, sintetizador (simples e configurável), reprodutor de áudio, consulta ao histórico, preferências, arquivo do banco (mudar de lugar e copiar), persistência completa, lançador de programas, dispositivos de áudio, inicialização com o sistema e catálogo de modelos de linguagem.	Domínio

Camada	Módulos e responsabilidade	Pode depender de
Aplicação (`aplicacao`)	Casos de uso e políticas: `orquestrador` (ciclo de conversa), `pipeline` (texto em fala, em fluxo contínuo), `vigia` (escuta passiva), `sessao` (políticas de voz, idioma, ativação, microfone e saudação; mudança e backup do banco; modelos disponíveis), `memoria` (“o que eu fiz”), `programas`, `agendador` (lembretes), `barramento` (eventos entre threads) e `frases` (divisor incremental de frases).	Domínio, portas, `i18n` e `configuracao`
Adaptadores (`adaptadores`)	Tecnologias concretas: `claude` (modelo com ferramentas e catálogo de modelos), `ollama` (modelo local com respostas em camadas e catálogo dos modelos instalados), `ferramentas` (esquemas, validação e execução), `offline` (intenções locais), `sqlite` (memória de longo prazo), `microfone` (captura e reconhecimento), `ativacao_vosk`, `sintetizador` (edge-tts e gTTS com cache), `reprodutor` (pygame), `programas_linux` (entradas `.desktop`), `dispositivos_audio` (microfones do servidor de áudio), `inicializacao_linux` (abertura com o computador e comando no terminal), `alsa` (silenciamento de mensagens nativas) e `repositorio_json` (leitura do formato legado).	Domínio, portas, aplicação, `audio` e `i18n`
Núcleos de processamento (`audio`, `avatar`)	Processamento de sinais (tom, envelope de volume e timbre, detecção de fala, ganho automático, perfil de voz) e geração das imagens do avatar (catálogo, renderização paralela com cache e suavização da animação).	`configuracao` e `i18n`
Apresentação (`apresentacao`)	Interface Tkinter: `app` (janela principal), `palco` (avatar, anéis de estado e legendas), `cena` (canvas diferencial), `laco` (laço de quadros adaptativo e fila de trabalho com orçamento), `painel` (agenda), `configuracoes` (diálogo), `gravador` (ferramenta de gravação da voz) e `tema`.	Aplicação, domínio, portas, núcleos e raiz de composição
Raiz de composição (`bootstrap`)	Escolhe e conecta as implementações concretas; inicia os serviços lentos em paralelo.	Todas as camadas internas e os adaptadores
Transversais (`i18n`, `configuracao`)	Catálogo de mensagens e tradutor; configuração imutável lida do ambiente.	Nenhuma camada de infraestrutura

Fonte: elaboração própria (2026).

5.3 CONTRATOS ENTRE AS CAMADAS

Os contratos são definidos como protocolos estruturais (Levkivskyi; Lehtosalo; Langa, 2017). Um componente satisfaz um contrato quando possui os métodos e atributos exigidos, sem

precisar herdar de uma classe base. Isso permite que os testes usem dublês simples (Meszaros, 2007) e que a substituição de uma tecnologia, como trocar o serviço de síntese de voz, não afete o domínio nem a aplicação. Os contratos foram mantidos pequenos, conforme o princípio da segregação de interfaces. Por exemplo, o serviço de memória depende apenas da consulta ao histórico; a agenda, apenas do repositório da agenda; e a tela de configurações, apenas de um sintetizador cuja voz possa ser trocada.

O contrato de persistência da agenda pertence ao próprio domínio (`dominio/repositorios.py`), pois é o domínio que declara do que precisa. A infraestrutura, representada pela classe `BancoSqlite`, implementa esse contrato e também os de histórico, de preferências e de arquivo do banco. A porta `PersistenciaAssistente` combina os quatro contratos para os poucos pontos que precisam do conjunto, como o núcleo da sessão.

Os modelos de linguagem informam o provedor e o modelo em uso, e cada provedor oferece um catálogo (`CatalogoModelos`) com os modelos que o assistente consegue usar. Quando um catálogo não pode ser consultado, ele levanta uma exceção com a chave da mensagem no catálogo de textos, e não um texto pronto: o adaptador não precisa conhecer o idioma da interface.

5.4 MODELO DE DADOS

A persistência utiliza um único arquivo SQLite (`memoria.db`), cujo esquema é versionado por meio do mecanismo `PRAGMA user_version` e migrado automaticamente na abertura (SQLite, 2026). As tabelas são descritas no Quadro 10.

Quadro 10 – Tabelas do banco de dados

Tabela	**Conteúdo e restrições**
`tarefas`	Identificador, texto, data de criação, indicador de conclusão (restrito a 0 ou 1) e data de conclusão.
`lembretes`	Identificador, texto, data e hora do aviso e indicador de aviso já dado.
`historico`	Sequência, data e hora do fato, tipo do evento, entidade, identificador da entidade, descrição no momento do fato, dados adicionais em JSON, resumo do registro anterior e resumo do próprio registro (único). Índice por data. Gatilhos recusam qualquer UPDATE ou DELETE.
`preferencias`	Pares chave e valor com as escolhas da tela de configurações (avatar, voz, tom, ritmo, ativação por voz, idioma e microfone).
`meta`	Pares chave e valor de controle interno, como o próximo identificador.

Fonte: elaboração própria (2026).

Os tipos de evento registrados no histórico são: tarefa criada, tarefa concluída, tarefa reaberta, tarefa removida, lembrete criado, lembrete avisado e lembrete removido. Cada registro guarda a descrição do item no momento do fato, de modo que a resposta sobre o passado continua correta mesmo se o item for removido depois. O resumo de cada registro é calculado

com SHA-256 (National Institute of Standards and Technology, 2015) sobre o resumo anterior e os campos do registro; a verificação de integridade recalcula a cadeia e aponta o primeiro registro inconsistente, detectando alteração, remoção, inserção por fora do sistema e quebra de encadeamento.

O arquivo fica, por padrão, na pasta de dados, mas pode ser movido para outra pasta pela tela de configurações. Como o banco não pode guardar o próprio endereço, o local escolhido é registrado em um pequeno arquivo JSON na pasta de dados, lido na abertura. A mudança e o backup usam a interface de backup do SQLite (SQLite, 2026), que produz uma cópia consistente mesmo com gravações em andamento; a cópia é conferida com `PRAGMA integrity_check` antes de ser aceita, e o arquivo anterior é renomeado com a data, nunca apagado.

5.5 MODELO DE CONCORRÊNCIA

Um assistente de voz precisa ouvir, pensar, falar e animar ao mesmo tempo, mas a biblioteca Tkinter não é segura para uso por várias threads. A solução adotada combina quatro mecanismos:

a) **barramento de eventos**: as threads de trabalho publicam eventos imutáveis (mudança de estado, fala reconhecida, resposta parcial, nível do microfone, aviso, serviços prontos, avatar pronto) em uma fila segura; a thread principal os despacha a cada quadro da animação, e somente ela altera componentes gráficos;

b) **uma interação por vez**: uma trava garante que apenas uma interação (ouvir, pensar e falar) esteja em andamento; anúncios, como saudação e lembretes, entram em uma fila e são falados entre as interações;

c) **token de cancelamento por interação**: um objeto `threading.Event` novo é criado de forma síncrona no início de cada interação; todos os estágios consultam esse token, de modo que uma interrupção nunca se perde e estágios cancelados anteriormente não são reativados;

d) **máquina de estados explícita**: as transições válidas são declaradas (por exemplo, de “ouvindo” só é possível ir para “parado” ou “pensando”); uma transição inválida gera erro imediato, em vez de um defeito silencioso.

As threads existentes em execução normal são: a principal (interface e animação); a de inicialização dos serviços; as de renderização do avatar e de geração dos halos; a da interação corrente; as dos estágios “pensar” e “sintetizar” do pipeline, com duas threads de síntese de voz; a da escuta passiva; e a do agendador de lembretes, que dorme até o próximo vencimento.

Como só a thread principal pode criar imagens para a biblioteca gráfica, a conversão das imagens preparadas em segundo plano (quadros do avatar e halos) passa por uma fila de trabalho com orçamento de 4 ms por quadro. Cada tarefa é um gerador que converte uma imagem e devolve o controle, de modo que um trabalho de centenas de milissegundos se espalha por vários quadros sem interromper a animação.

5.6 FLUXOS DE EXECUÇÃO

5.6.1 Inicialização

1. O ponto de entrada lê as opções da linha de comando e a configuração imutável do ambiente.
2. A raiz de composição monta o núcleo, uma operação instantânea: abre o banco de dados no local escolhido ou no padrão (migrando a agenda JSON antiga, se houver), cria a agenda, carrega as preferências, define o idioma e cria os serviços de memória e de programas.
3. A janela é exibida imediatamente, com tamanho fixo, e o laço de quadros começa; ele roda a 60 quadros por segundo com o assistente ativo e a 30 em repouso.
4. Em segundo plano, o avatar é renderizado ou lido do cache, os halos são gerados, e os serviços lentos são iniciados em paralelo: conexão com o modelo de linguagem, tentando o Claude, depois o modelo local (Ollama) e, por fim, o modo offline; calibração do microfone; abertura do áudio; carregamento do modelo de ativação; e leitura da lista de programas instalados.
5. Quando os serviços ficam prontos, um evento avisa a interface, que habilita o botão de microfone, inicia o agendador, enfileira a saudação e liga a escuta passiva.

5.6.2 Conversa em pipeline

1. O usuário pressiona ESPAÇO, diz “ok Fael” ou envia um texto; o orquestrador adquire a trava de interação, libera o microfone da escuta passiva e cria um token de cancelamento.
2. No estado “ouvindo”, o reconhecedor capta a fala com detecção de atividade de voz e devolve o texto; o nível do microfone é publicado continuamente para a interface.
3. No estado “pensando”, o modelo começa a transmitir a resposta em pedaços de texto.
4. O estágio “pensar” do pipeline acumula os pedaços, publica a resposta parcial para a legenda e entrega frases completas a uma fila; o divisor de frases corta a primeira frase cedo e não quebra números nem abreviações.
5. O estágio “sintetizar” pede a síntese de cada frase assim que ela fica pronta, com até duas sínteses simultâneas; a thread que toca decodifica o áudio, apara o silêncio das pontas (a voz sintetizada traz cerca de 1 s de silêncio ao fim de cada frase) e calcula o envelope de volume e de timbre que comandará a boca do avatar.
6. A thread da interação toca as faixas em ordem, no estado “falando”, enquanto as frases seguintes ainda estão sendo geradas e sintetizadas.
7. Ao fim, a interação publica a resposta concluída, volta ao estado “parado”, fala os anúncios pendentes e libera a trava; a escuta passiva retoma o microfone.

Esse arranjo faz com que a espera percebida seja a da primeira frase, e não a da resposta

inteira. Uma interrupção sinaliza o token: a reprodução para, as filas são esvaziadas sem sintetizar e o fluxo de texto do modelo é fechado, o que descarta a rodada no histórico da conversa.

5.6.3 Ativação por voz

1. Enquanto o assistente está parado, a vigia reserva o microfone e entrega o áudio ao Vosk, que só pode escolher entre as variantes aceitas de “ok Fael”, frases distratoras parecidas (como “o que foi” e “ok Rafael”) e um marcador de fala desconhecida.
2. O áudio passa por um controle automático de ganho, que amplifica a voz até um nível adequado, com teto de 8 vezes e resistência a picos isolados; se a voz chega fraca demais de forma persistente, um aviso é exibido.
3. Ao reconhecer uma variante aceita, a vigia continua gravando enquanto houver voz, para capturar um comando dito na mesma frase.
4. Se houver comando, ele é transcrito com precisão pelo reconhecedor remoto e alinhado com a transcrição local, para remover a frase de ativação mesmo quando ela for mal transcrita; se não houver, um sinal sonoro de dois tons confirma a ativação e o assistente passa a ouvir.
5. Em ambos os casos, a interação segue o fluxo de conversa descrito anteriormente.

5.6.4 Memória de longo prazo

1. Cada alteração na agenda gera, na mesma transação, um ou mais registros de auditoria.
2. Quando o usuário pergunta “o que eu fiz…”, o período é interpretado por uma lista ordenada de interpretadores, do mais específico para o mais genérico, sempre voltada ao passado.
3. O serviço de memória consulta os registros do período e calcula o resultado líquido de cada tarefa: concluídas, criadas e ainda pendentes, e lembretes avisados.
4. A resposta é falada no idioma configurado; com o modelo de linguagem ativo, a mesma consulta está disponível como ferramenta.

5.6.5 Abertura de programas

1. O lançador lê as entradas `.desktop` dos diretórios padronizados pelo freedesktop.org (Freedesktop.org, 2020), inclusive de Snap e Flatpak, ignorando entradas ocultas e as que exigem programas ausentes.
2. O pedido é extraído da fala (“abre o navegador”), normalizado e comparado com nomes, palavras-chave e sinônimos em português e inglês; “navegador” resolve para o navegador padrão do sistema.

3. Se houver um candidato claro, o programa é aberto sem interpretador de comandos; se houver empate, o assistente pergunta qual das opções o usuário deseja.

5.6.6 Laço de ferramentas do modelo de linguagem

1. A fala é acrescentada ao histórico da conversa com a data e a hora atuais, mantendo as instruções de sistema estáveis e, portanto, reaproveitáveis pelo cache do serviço.
2. O texto da resposta é repassado ao pipeline conforme chega.
3. Ao fim de cada rodada, se o modelo pediu ferramentas, cada entrada é validada contra o esquema e executada; os resultados são devolvidos ao modelo, até o limite de oito rodadas.
4. Recusas, entradas truncadas pelo limite de tamanho, erros da API e interrupções descartam a rodada inteira do histórico e produzem uma frase explicativa para o usuário.

5.6.7 Respostas do modelo local

1. Se a fala é um comando que o modo offline reconhece, ele responde, sem consultar o modelo.
2. Se a fala trata de agenda, lembretes ou histórico, o assistente diz “só um instante” e envia o pedido ao modelo apenas com as ferramentas do assunto que criam ou consultam, nunca as que concluem, removem ou abrem programas.
3. Caso contrário, a fala é conversa e vai ao modelo sem ferramentas, com o texto transmitido em fluxo contínuo para o pipeline.
4. Uma chamada de ferramenta escrita como texto pelo modelo é executada, se for válida, ou descartada; ela nunca é falada.
5. Se o servidor local não responde, a fala é atendida pelo modo offline, e o modelo só é tentado de novo depois de 30 s.

5.6.8 Animação da boca

1. Antes de tocar, cada faixa tem seu envelope calculado quadro a quadro: o volume define a abertura, e a razão entre a energia das frequências agudas (1 a 3 kHz) e graves (250 Hz a 1 kHz) define o formato da boca (arredondada para “o” e “u”, neutra para “a” e aberta para “e” e “i”).
2. Durante a reprodução, o reprodutor expõe a amplitude e o formato do instante atual.
3. A cada quadro, o palco suaviza esses valores de forma independente da taxa de quadros e escolhe a imagem pré-renderizada correspondente, entre 36 combinações (três formatos e doze níveis).
4. O palco descreve a cena inteira a cada quadro, mas um canvas diferencial só repassa

à biblioteca gráfica as coordenadas e opções que mudaram, já que cada alteração obriga a redesenhar a área afetada, inclusive as imagens grandes que estão por baixo.

5.7 DECISÕES DE PROJETO

O Quadro 11 resume as principais decisões de projeto e suas justificativas.

Quadro 11 – Principais decisões de projeto

Nº	**Decisão**	**Justificativa**
1	Arquitetura hexagonal com protocolos	Troca de tecnologia sem tocar no domínio; dublês triviais nos testes.
2	Pipeline de três estágios com filas	A latência percebida passa a ser a da primeira frase.
3	Um token de cancelamento por interação, criado de forma síncrona	Interrupção instantânea, que nunca se perde nem reativa estágios antigos.
4	Barramento de eventos despachado no laço de quadros	A interface só é alterada pela thread principal.
5	Máquina de estados explícita	Estados impossíveis tornam-se erros visíveis.
6	Histórico da conversa somente de acréscimo, com descarte por rodada	O histórico nunca fica inconsistente, mesmo com interrupções.
7	Validação local das entradas das ferramentas	Entradas transmitidas em fluxo podem chegar truncadas; nada inválido é executado.
8	Detecção de fala própria sobre o microfone	Medidor de nível real, cancelamento e pausa de fim de fala menor.
9	Reconhecimento local e restrito da frase de ativação	Privacidade e exatidão em frases curtas.
10	SQLite com histórico somente de acréscimo e cadeia de resumos	Memória confiável e auditável em um único arquivo local.
11	Renderização prévia do avatar com cache invalidado pelo resumo do código	A animação apenas troca imagens; o cache nunca fica desatualizado.
12	Testes de arquitetura	A regra de dependência e os princípios SOLID não dependem apenas de disciplina.
13	Despacho por tabelas (intenções, ferramentas e períodos)	Baixa complexidade e extensão sem editar o código existente.
14	Canvas diferencial e halo opaco	O redesenho de imagens com transparência dominava o custo; enviar só o que muda e evitar transparência reduziu o repouso de 29% para 5% de um núcleo.
15	Taxa de quadros adaptativa e fila de trabalho com orçamento	Fluidez quando há movimento, economia em repouso e nenhum engasgo ao converter imagens.
16	Corte do silêncio e síntese paralela de frases	As frases se emendam como na fala natural e a primeira palavra sai antes.
17	Modelo local em camadas, com ferramentas seguras	Um modelo pequeno conversa bem, mas escolhe ferramentas mal; ações destrutivas ficam com comandos determinísticos.

Nº	Decisão	Justificativa
18	Microfone escolhido pelo servidor de áudio, com variável definida só durante a abertura	Evita disputar o dispositivo com o PipeWire e não afeta os programas abertos pelo assistente.
19	Estado da inicialização lido dos próprios arquivos	A tela nunca contradiz o sistema, mesmo se o usuário mexer nos atalhos por fora.
20	Mudança do banco por cópia conferida, sem apagar o anterior	Nenhuma operação sobre os dados do usuário é destrutiva ou irreversível.

Fonte: elaboração própria (2026).

6 CONSTRUÇÃO DO SISTEMA: PASSO A PASSO

Este capítulo relata, em ordem cronológica, os vinte incrementos que levaram o Robofael da primeira versão funcional à versão 3.8.0. Para cada incremento são descritos o pedido que o motivou, o que foi construído, as decisões tomadas, os problemas encontrados e a forma de verificação. A Tabela 1 resume a evolução.

Tabela 1 – Evolução do sistema por incremento

Nº	Versão	Incremento	Testes
1	–	Protótipo: avatar a partir das fotos, voz, agenda e modelo de linguagem	–
2	–	Busto centralizado e boca suave	–
3	–	Perfil de voz (ajuste de tom e ritmo)	–
4	–	Nitidez, camisa futurista e boca natural	–
5	1.0.0	Estrutura profissional, documentação e testes unitários	98
6	2.0.0	Refatoração completa: arquitetura hexagonal e pipeline em fluxo contínuo	168
7	2.1.0	Renomeação para Robofael	168
8	–	Gravação da voz do usuário e criação do perfil real	168
9	2.2.0	Ativação por voz local (“ok Robofael”)	211
10	2.3.0	Nova frase de ativação (“ok Fael”) com gramática restrita	247
11	3.0.0	Memória de longo prazo, “o que eu fiz”, janela fixa, configurações e programas	351
12	3.0.1	Ganho automático do microfone e aviso de voz baixa	357
13	3.1.0	Internacionalização (português e inglês)	508
14	3.2.0	Instalador padrão para Ubuntu	508
15	3.3.0	Revisão SOLID e código limpo, com testes de arquitetura	529
16	3.4.0	Desempenho e fluidez guiados por medição; tipagem completa	546
17	3.5.0	Nome técnico `robofael`, com compatibilidade com os nomes antigos	549
18	3.6.0	Modelo de linguagem local (Ollama) com respostas em camadas	576
19	3.7.0	Escolha do microfone e abertura junto com o computador	601
20	3.8.0	Local e backup do banco; modelo em uso e modelos compatíveis	643

Fonte: elaboração própria (2026), a partir do registro de mudanças do projeto.

6.1 INCREMENTO 1: PROTÓTIPO FUNCIONAL

Pedido. Criar um programa em Python que manipulasse as imagens do usuário, presentes na pasta do projeto, para transformá-las no avatar de um assistente pessoal de atividades diárias que ouve e responde por voz.

Construção. Foram usadas duas fotografias: a do crachá e uma *selfie*, ambas já sem fundo. Em cada uma foram medidas manualmente as posições da boca e do queixo. A animação foi obtida pela deformação suave da região da mandíbula para baixo, com o preenchimento do espaço aberto por um interior escuro de boca, em oito níveis de abertura. Um brilho ciano nas bordas deu ao avatar a aparência de assistente digital. A voz foi implementada com reconhecimento de fala em português pelo serviço do Google e síntese pela voz neural `pt-BR-AntonioNeural`, com o gTTS como reserva. A abertura da boca passou a acompanhar o volume real do áudio, quadro a quadro. A agenda de tarefas e lembretes falados era salva em um arquivo JSON, e o modelo Claude, quando disponível, operava a agenda por meio de ferramentas; sem chave de acesso, um modo offline atendia a comandos básicos.

Verificação. A interface foi executada com os dois avatares e a fala sincronizada com a animação. A integração com o modelo de linguagem não pôde ser exercitada de imediato, por falta de credenciais na máquina, e foi registrada como pendente.

6.2 INCREMENTO 2: BUSTO CENTRALIZADO E BOCA SUAVE

Pedido. Exibir apenas um busto, centralizado, e suavizar a animação da boca.

Construção. Mediu-se o topo e o centro da cabeça em cada foto, e o recorte passou a ir de um pouco acima da cabeça até os ombros, centralizado na cabeça, e não na foto inteira. Com isso, a *selfie*, que estava deslocada para o lado, ficou centralizada. Uma moldura oval com bordas suaves passou a esconder os cortes das fotos. Para a boca, o número de aberturas subiu de 8 para 16, mais próximas entre si quando a boca está quase fechada, onde os saltos eram mais visíveis. Cada quadro passou a ser desenhado no dobro do tamanho e depois reduzido (superamostragem), eliminando o serrilhado. O volume da voz passou a ser lido 60 vezes por segundo, sem picos isolados nem ruído de fundo. A boca passou a abrir rápido e fechar devagar, como na fala real, e a animação foi para cerca de 60 quadros por segundo.

Verificação. Capturas de tela confirmaram o enquadramento e a passagem por aberturas intermediárias durante a fala.

6.3 INCREMENTO 3: PERFIL DE VOZ

Pedido. Registrar a voz do usuário e fazer o assistente falar com o mesmo tom. Diante das alternativas, ajustar apenas o tom de uma voz neural ou clonar o timbre localmente, o usuário escolheu a primeira.

Construção. Foi criado um gravador de voz com interface própria, que apresenta frases para leitura, detecta o fim de cada uma e permite regravar. Com pelo menos três gravações, o programa estima o tom médio por autocorrelação e a velocidade de fala. Em seguida, escolhe a voz neural em português de tom mais próximo e ajusta tom e ritmo até que a voz sintética fique a menos de 3 Hz da voz do usuário. O perfil é salvo em arquivo JSON.

Verificação. O medidor de tom acertou sinais de teste com erro máximo de 1 Hz. Em

uma simulação com um “usuário” 25 Hz mais grave e 15% mais lento, o perfil resultou em um ajuste de -22 Hz, com o tom final a 2 Hz do alvo, e em um ritmo de -14%.

6.4 INCREMENTO 4: NITIDEZ, CAMISA FUTURISTA E BOCA NATURAL

Pedido. Aumentar a nitidez do avatar, trocar a roupa por uma camisa de corte futurista, tornar o movimento da boca mais natural e manter o busto centralizado.

Construção. Constatou-se que a *selfie* sem fundo tinha apenas 433 por 577 pixels, mas o arquivo original tinha 1200 por 1600 pixels na mesma proporção. Passou-se então a usar a imagem sem fundo apenas como máscara de contorno, aplicada sobre a foto em alta resolução. A foto do crachá, disponível apenas em 500 pixels, recebeu ampliação de alta qualidade, filtro de nitidez e leve aumento de contraste. A camisa original foi substituída por uma camisa azul-grafite desenhada por cima, com gola alta metálica de borda luminosa, costuras em V com brilho ciano, placas nos ombros com pequenos LEDs e um emblema no peito. As dobras do tecido foram preservadas aproveitando-se a luz e a sombra da camisa original. A boca deixou de ser um corte que descia e passou a ser uma deformação contínua: o lábio superior sobe levemente, a mandíbula desce, os cantos ficam presos e a abertura tem formato de lente, com sombra sob os dentes e língua visível nas aberturas maiores. Foram introduzidos três formatos de vogal (arredondado para “o” e “u”, neutro para “a” e mais largo para “e” e “i”), escolhidos a partir do timbre do áudio.

Verificação. Prévias geradas com as mesmas imagens e posições da interface, além da execução da interface com fala simulada, sem erros.

6.5 INCREMENTO 5: ESTRUTURA PROFISSIONAL E TESTES (VERSÃO 1.0.0)

Pedido. Organizar o projeto com estrutura profissional, pronta para versionamento, com um arquivo README que explicasse a arquitetura, a forma de execução e os fluxos, e com testes unitários.

Construção. Os scripts soltos foram reorganizados em um pacote Python instalável, com subpacotes para lógica (agenda, conversa, ferramentas, modo offline), áudio, avatar e interface, e com a configuração centralizada e ajustável por variáveis de ambiente. As fotos foram movidas para uma pasta de recursos com nomes legíveis. A lógica foi separada do microfone, do alto-falante e da tela, e o cliente do modelo, o relógio e as pastas tornaram-se substituíveis por versões simuladas, o que viabilizou os testes sem hardware. Os dados pessoais (agenda, perfil e gravações) e as imagens geradas foram colocados em pastas excluídas do controle de versão. Foram criados um Makefile, o arquivo de projeto `pyproject.toml` e o README.

Verificação. 98 testes unitários aprovados em cerca de três segundos e análise de estilo sem ocorrências.

6.6 INCREMENTO 6: REFATORAÇÃO COMPLETA (VERSÃO 2.0.0)

Pedido. Refatorar o projeto do início ao fim, com nível de engenharia de grandes empresas de tecnologia, aperfeiçoando a usabilidade, o desempenho e a organização arquitetural.

Diagnóstico. Antes de qualquer alteração, foi feito um diagnóstico. O fluxo era totalmente serial: esperava-se a resposta inteira do modelo, sintetizava-se todo o áudio e só então a fala começava, com três a seis segundos de silêncio. O microfone usava uma função da biblioteca que funcionava como caixa-preta, sem nível de áudio nem cancelamento. Os estados eram cadeias de caracteres alteradas por várias threads. A renderização do avatar bloqueava a abertura da janela. Os lembretes eram verificados a cada 15 segundos. A escrita direta do JSON podia corromper o arquivo. Por fim, a classe principal da interface conhecia todas as tecnologias.

Construção. A solução foi reescrita em arquitetura hexagonal, com:

a) domínio tipado com classes de dados no lugar de dicionários;
b) máquina de estados explícita e barramento de eventos;
c) pipeline em fluxo contínuo de três estágios, com divisor incremental de frases;
d) áudio decodificado em memória e envelope da boca calculado antes da reprodução;
e) cache em disco das falas sintetizadas;
f) renderização paralela do avatar fora da thread da interface, com cache invalidado pelo resumo do código;
g) inicialização paralela dos serviços em segundo plano;
h) agendador que dorme até o próximo lembrete;
i) detecção de fala própria, com medidor de nível real e pausa de fim de fala de 0,7 s;
j) escrita atômica da agenda e preservação de arquivos corrompidos;
k) histórico da conversa somente de acréscimo, com descarte por rodada;
l) validação local das entradas das ferramentas.

Foram adotados o verificador de tipos, a análise estática, os ganchos de *pre-commit* e a integração contínua com servidor gráfico virtual.

Problemas corrigidos. Foram corrigidos cinco problemas: um cancelamento que se perdia quando o ESPAÇO era pressionado duas vezes rapidamente; uma notificação perdida no agendador; um erro de ponto flutuante no limiar de tempo da detecção de fala; o acesso à interface do gravador fora da thread principal; e valores padrão da configuração incompatíveis com classes de dados compactas.

Verificação. 168 testes, incluindo um teste que prova que a primeira frase toca antes de o modelo terminar de gerar a resposta. A janela passou a aparecer em 0,8 s, e a primeira palavra no modo offline, em cerca de 1,3 s.

6.7 INCREMENTO 7: RENOMEAÇÃO PARA ROBOFAEL (VERSÃO 2.1.0)

Pedido. Mudar o nome do assistente de EuRobô para Robofael.

Construção. O nome passou a ficar centralizado em um único campo da configuração, ajustável por variável de ambiente, e foi aplicado ao título da janela, à saudação, às instruções do modelo, à ferramenta de gravação, aos registros e à documentação. Optou-se por manter o nome técnico do pacote (`eurobo`) para não quebrar importações, comandos e caminhos existentes. A renomeação técnica para `robofael` só ocorreu na versão 3.5.0 (Incremento 17), mantendo os comandos e as variáveis de ambiente antigos como alternativas.

Verificação. Testes do nome padrão, da troca por variável e da apresentação do modelo como Robofael.

6.8 INCREMENTO 8: GRAVAÇÃO DA VOZ REAL DO USUÁRIO

Pedido. Gravar a voz do usuário.

Construção e resultado. A ferramenta de gravação foi aberta, e o usuário gravou as cinco frases. O tom médio medido foi de 116 Hz. A voz base mais próxima foi a `pt-BR-AntonioNeural`, com 137 Hz. O ajuste de tom de -21 Hz levou a voz sintética a 118 Hz, a 2 Hz da voz do usuário, e o ritmo ficou em -26%. As gravações tiveram entre 3,9 e 5,2 segundos de fala cada uma, com tom entre 95 e 119 Hz, e o perfil usa a mediana das cinco para reduzir o efeito de frases interrogativas.

Problemas e diagnóstico. Durante a primeira execução real, o modelo de linguagem não conectou. A investigação mostrou que a configuração do terminal do usuário direcionava as chamadas da API para um servidor local de outro fornecedor, onde o modelo não existia. A mensagem de aviso foi aprimorada para indicar essa causa, sem alterar a configuração pessoal do usuário. Também foi eliminado o ruído de mensagens nativas do servidor de áudio JACK que aparecia no terminal a cada abertura do microfone.

6.9 INCREMENTO 9: ATIVAÇÃO POR VOZ (VERSÃO 2.2.0)

Pedido. Acionar a captura de voz com a frase “ok Robofael”, em vez de pressionar o ESPAÇO. Entre o reconhecimento local e o remoto da frase de ativação, o usuário escolheu o local.

Construção. Foi implementada a escuta passiva local com o Vosk e um modelo compacto de 31 MB para português, para que o áudio do ambiente não deixasse o computador. Como o modelo não conhece a palavra “Robofael” e a transcreve por aproximação (por exemplo, “o que roubou fel”), a detecção foi feita por semelhança fonética, calibrada com transcrições reais: ativa com a frase sozinha, com a frase seguida de comando e com a frase dita após outra conversa, mas não com “ok Rafael” nem com conversas comuns. Quando o reconhecedor remoto erra a frase de ativação, como em “Ok Google Fael, me lembra…”, o comando é recuperado pelo alinhamento das duas transcrições. Um sinal sonoro de dois tons confirma a ativação.

Problemas corrigidos. A biblioteca de áudio capturava os sinais de interrupção do

sistema, e o programa não fechava com Ctrl+C nem com SIGTERM. A solução foi desativar os tratadores de sinal da biblioteca e tratar os sinais no laço da interface.

Verificação. 211 testes; reconhecimento real exercitado com a voz neural do perfil injetada no lugar do microfone.

6.10 INCREMENTO 10: FRASE “OK FAEL” E GRAMÁTICA RESTRITA (VERSÃO 2.3.0)

Pedido. Trocar a frase de ativação para “ok Fael”.

Construção. A palavra “Fael”, curta e fora do vocabulário do modelo local, era simplesmente descartada: “ok Fael” virava “ok” ou “o que foi”. A transcrição livre foi então substituída por uma gramática restrita, na qual o reconhecedor só pode escolher entre variantes aceitas de “ok Fael” e frases distratoras parecidas e frequentes. Com isso, o que não é a frase de ativação é absorvido pelas distratoras. A frase de ativação passou a ser configurável e independente do nome do assistente. Como a pausa da vírgula dividia a fala em duas partes, a escuta passou a continuar gravando enquanto houvesse voz após a frase. O comando também passou a ser limpo da frase quando o reconhecedor remoto a transcreve de forma inesperada, como em “o que Israel anota tarefa…”, que resulta em “anota tarefa…”.

Verificação. Calibração com cinco vozes sintéticas: zero falsos positivos em 115 frases parecidas e 8 acertos em 8 tentativas na voz do perfil do usuário; 247 testes.

6.11 INCREMENTO 11: MEMÓRIA DE LONGO PRAZO E OUTRAS FUNÇÕES (VERSÃO 3.0.0)

Pedido. Adicionar um banco SQLite como memória de longo prazo, com histórico auditável de tarefas e lembretes, para que o assistente respondesse o que o usuário fez em uma data ou semana do passado. O pedido incluía ainda tornar fixo o tamanho da janela, porque a tela cheia quebrava a renderização; criar um botão de configuração de avatar e voz; permitir abrir programas instalados no Linux; e garantir a integridade de todos os testes.

Construção. O pedido foi dividido em seis frentes:

a) **memória em SQLite**: agenda, histórico e preferências em um único arquivo; histórico somente de acréscimo, com gatilhos que recusam alteração e remoção e com cadeia de resumos SHA-256; estado e histórico gravados na mesma transação; esquema versionado; importação automática do arquivo JSON anterior, renomeado e preservado; verificação de integridade por comando;

b) **“o que eu fiz”**: interpretação de períodos (dezenove formas testadas) e cálculo do resultado líquido no período, disponível no modo offline e como ferramenta do modelo;

c) **janela de tamanho fixo**: tamanhos mínimo e máximo iguais e redimensionamento desativado;

d) **configurações**: diálogo com avatar, voz, tom, ritmo, botão para ouvir, uso do perfil

gravado e ativação por voz, com cancelamento que desfaz o teste e preferências persistidas;

e) **programas**: leitura das entradas `.desktop`, sinônimos, navegador padrão, desambiguação e abertura sem interpretador de comandos;

f) **testes**: cobertura de todas as frentes, incluindo a detecção de quatro tipos de adulteração do histórico.

Problemas corrigidos. Um teste revelou que “anota tarefa abrir conta no banco” abria um programa, em vez de criar a tarefa; os comandos de agenda passaram a ter prioridade. A análise estática apontou uma função anônima que usava uma variável já descartada pelo tratamento de exceção. As variáveis da interface eram destruídas fora da thread principal pelo coletor de lixo e passaram a ser liberadas explicitamente.

Verificação. 351 testes aprovados em três execuções seguidas e execução real com uma cópia dos dados do usuário, na qual a migração, a consulta ao histórico e a verificação de integridade funcionaram.

6.12 INCREMENTO 12: GANHO AUTOMÁTICO (VERSÃO 3.0.1)

Contexto. Os primeiros testes com a voz real do usuário mostraram que a frase de ativação não era reconhecida: o reconhecedor ouvia apenas “ok” e “que”, e o pico do sinal ficava em cerca de 580, quando o necessário era superior a 1000. A investigação, feita somente por leitura, revelou que o microfone em uso era o interno do computador, com entrada em 37%, enquanto um microfone USB conectado não estava sendo usado.

Construção. Foi implementado um controle automático de ganho que amplifica até 8 vezes a voz que chega baixa, resistente a picos isolados, e um aviso de voz baixa no terminal e na tela, com orientação ao usuário. As configurações de áudio do sistema não foram alteradas; os comandos para ajustá-las foram fornecidos ao usuário.

Verificação. 357 testes; após o ajuste, os registros mostraram nove ativações bem-sucedidas com a voz real do usuário.

6.13 INCREMENTO 13: INTERNACIONALIZAÇÃO (VERSÃO 3.1.0)

Pedido. Permitir que o assistente responda em inglês quando configurado.

Construção. Todas as mensagens foram reunidas em um catálogo único, com as duas línguas lado a lado, e um tradutor foi injetado em todos os componentes, com troca ao vivo. O idioma passou a afetar as respostas, a interface, a voz neural (Antonio em português; Andrew, Brian, Guy ou Jenny em inglês), a voz de reserva, o reconhecimento de fala e as instruções do modelo, cuja conversa recomeça na troca. Os comandos passaram a ser entendidos nos dois idiomas, independentemente do idioma das respostas, e os períodos em inglês foram suportados, com datas numéricas seguindo a convenção de cada idioma.

Problemas corrigidos. O diálogo de configurações podia falhar ao se tornar modal antes

de aparecer na tela e passou a tentar novamente. Além disso, as instruções ao modelo deixaram de atribuir gênero ao usuário.

Verificação. 508 testes, inclusive um que garante que nenhuma tradução está faltando e que os campos de formatação coincidem nas duas línguas; execução real em inglês com uma cópia dos dados.

6.14 INCREMENTO 14: INSTALADOR PARA UBUNTU (VERSÃO 3.2.0)

Pedido. Criar um instalador padrão para ambientes Ubuntu.

Construção. Optou-se por um pacote `.deb`, o formato nativo da distribuição (Debian, 2026). O pacote contém o código, as fotos, o modelo de ativação e todas as dependências Python em formato *wheel*, o que permite instalar sem internet e de forma determinística. As dependências de sistema, como Python, Tk, PyAudio e PortAudio, são declaradas para o gerenciador de pacotes. Na instalação, um ambiente Python isolado é criado em `/opt/robofael` e verificado. O pacote fornece os comandos `robofael` e `robofael-gravar-voz`, atalhos no menu de aplicativos e um ícone gerado a partir do próprio avatar. Os dados de cada usuário ficam em `~/.local/share/robofael` e o cache em `~/.cache/robofael`, conforme as convenções XDG. A remoção apaga o programa, mas preserva os dados do usuário. A janela passou a declarar a classe `robofael`, para que o ambiente gráfico a associe ao ícone do menu.

Problemas corrigidos. Uma das dependências não era distribuída como *wheel* e passou a ser construída localmente. Além disso, o ambiente isolado reutilizava uma versão da biblioteca de imagens do sistema sem suporte à interface gráfica, o que foi resolvido pela instalação forçada das versões incluídas no pacote.

Verificação. Um roteiro de teste instala o pacote em uma pasta temporária e verifica a integridade dos arquivos, os atalhos, a criação do ambiente sem internet, os lançadores, os caminhos de dados, o funcionamento do núcleo e a remoção. A janela foi aberta a partir dessa instalação simulada.

6.15 INCREMENTO 15: REVISÃO SOLID E CÓDIGO LIMPO (VERSÃO 3.3.0)

Pedido. Validar o projeto de ponta a ponta, implementando e verificando os conceitos SOLID e de código limpo, garantindo que os testes continuassem funcionando.

Construção. Esse incremento é detalhado no Capítulo 7. Em síntese, foram feitas as seguintes alterações:

a) o orquestrador foi dividido em três classes;
b) as políticas de sessão saíram da raiz de composição e da interface;
c) três despachos condicionais tornaram-se tabelas extensíveis;
d) contratos foram criados ou movidos para que todas as dependências apontassem para dentro;
e) uma violação do princípio de Liskov foi eliminada;

f) funções longas foram decompostas;
g) regras de complexidade foram incorporadas à análise estática;
h) testes de arquitetura passaram a verificar as regras automaticamente.

Problema corrigido. Durante a revisão, um teste de interface passou a falhar de forma intermitente apenas na suíte completa. A investigação, feita com a captura das pilhas de execução no momento da falha, revelou que a thread de renderização do avatar ficava bloqueada no destrutor de uma imagem Tk de um teste anterior: o coletor de lixo do Python, acionado naquela thread, chamava a biblioteca gráfica fora da thread principal. O defeito era latente e havia sido apenas exposto pela mudança no momento da coleta. A correção recolhe esses objetos na thread principal ao fim de cada teste de interface, e a fixture duplicada foi unificada.

Verificação. 529 testes, 85% de cobertura, nenhuma função com complexidade ciclomática acima de 10 e imagens do avatar idênticas, pixel a pixel, às anteriores à refatoração.

6.16 INCREMENTO 16: DESEMPENHO E FLUIDEZ GUIADOS POR MEDIÇÃO (VERSÃO 3.4.0)

Pedido. Refatorar novamente o projeto de ponta a ponta, aperfeiçoando a usabilidade e o desempenho, com uma organização ainda mais moderna e profissional.

Diagnóstico. Como as revisões anteriores já haviam tratado a estrutura, este incremento começou por medições do aplicativo real, e não pela leitura do código. Com o assistente parado, depois da saudação, o processo consumia 29% de um núcleo de processamento. Um microbenchmark do palco mostrou que cada quadro levava cerca de 13 ms, perto do limite de 16,6 ms de um quadro a 60 por segundo. Medições isoladas de cada elemento revelaram a causa: qualquer alteração sob o halo (600 por 600 pixels com transparência) ou sob o avatar obrigava a biblioteca gráfica a refazer, em software, a mistura de transparência de toda a área, e girar um único arco custava 8 ms. Além disso, a montagem dos 40 halos bloqueava a thread da interface por 0,75 s antes de a janela aparecer, e a troca de avatar convertia 36 imagens de uma vez, com um engasgo de cerca de 170 ms. Na fala, a medição do áudio sintetizado mostrou que cada frase trazia cerca de 0,25 s de silêncio no início e de 0,9 a 1,2 s no fim, o que, na reprodução frase a frase, se transformava em pausas artificiais.

Construção. Foram realizadas as seguintes alterações:

a) **canvas diferencial**: uma camada entre o palco e a biblioteca gráfica guarda o último valor enviado de cada item e só repassa coordenadas, em pixels inteiros, e opções que mudaram;
b) **halo opaco**: o halo passou a ser gerado já misturado com a cor de fundo, de modo que a biblioteca copia os pixels em vez de misturar transparência; o redesenho sob o halo caiu de cerca de 8 ms para 0,1 ms;
c) **taxa de quadros adaptativa**: um laço de quadros extraído da janela principal roda a 60 quadros por segundo com o assistente ouvindo, pensando ou falando, ou durante

transições de cor, e a 30 em repouso, agendando pelo relógio para não acumular atraso;

d) **fila de trabalho com orçamento**: os halos passaram a ser gerados em segundo plano, e a conversão de halos e quadros do avatar para a biblioteca gráfica é feita aos poucos, com no máximo 4 ms por quadro;

e) **corte do silêncio**: uma função de processamento de sinal localiza o trecho com fala e mantém margens de 30 ms antes e 150 ms depois, preservando consoantes fracas no início;

f) **síntese paralela**: até duas frases são sintetizadas ao mesmo tempo e tocadas na ordem original; a decodificação continua em uma única thread, porque o mixer de áudio não é feito para uso concorrente;

g) **pedido digitado durante a fala**: enviar um texto com o assistente ocupado passou a interromper o que ele fazia e atender o pedido novo, em vez de descartá-lo; se vários forem enviados, vale o mais recente, e uma trava garante que o pedido seja atendido exatamente uma vez;

h) **tipagem completa**: todas as funções passaram a ter assinatura tipada, exigência agora verificada pelo mypy.

Problemas evitados. Ao tipar a função de recorte do busto, uma primeira versão passou a usar coordenadas arredondadas também no cálculo da posição da boca, o que a deslocaria em até meio pixel. A comparação do resumo SHA-256 de todos os quadros, feita antes e depois, detectou a diferença, e a correção restaurou as imagens idênticas pixel a pixel. Uma medição inicial do consumo em repouso também se mostrou contaminada pela saudação falada e foi refeita depois que ela terminava, com a versão anterior medida nas mesmas condições.

Verificação. 546 testes, incluindo uma regressão de desempenho determinística: um palco em repouso, desenhado em um canvas falso que conta as chamadas, precisa enviar menos de 80 comandos por segundo, e as imagens grandes só podem ser redesenhadas poucas vezes por segundo. A cobertura ficou em 85,9%. No aplicativo real, o consumo em repouso caiu de 29% para 5% de um núcleo, a janela passou a aparecer em 0,6 s (antes, de 1,7 a 2,4 s), a primeira palavra da saudação passou a sair de 0,4 a 0,8 s mais cedo e a saudação completa ficou cerca de 1,6 s mais curta.

6.17 INCREMENTO 17: NOME TÉCNICO ROBOFAEL (VERSÃO 3.5.0)

Pedido. Renomear o nome técnico do projeto, que ainda era `eurobo`, para `robofael`, sem quebrar comandos e variáveis que o usuário já usasse.

Construção. O pacote Python, os comandos, as variáveis de ambiente, o lançador do pacote `.deb` e a documentação passaram a usar o nome `robofael`. Os nomes antigos foram mantidos como alternativas: os comandos `eurobo` e `eurobo-gravar-voz` continuam instalados como apelidos, e cada variável `EUROBO_*` ainda é lida quando a `ROBOFAEL_*`

correspondente não está definida, com um aviso de depreciação no registro. O lançador do pacote faz a mesma substituição, para que uma pasta de dados já configurada pelo nome antigo continue sendo usada.

Verificação. 549 testes, incluindo a leitura dos nomes antigos com aviso e a precedência do nome novo; um teste de arquitetura garante que o nome antigo só aparece no código de compatibilidade, e o roteiro do pacote confere que o lançador respeita a variável antiga da pasta de dados.

6.18 INCREMENTO 18: MODELO DE LINGUAGEM LOCAL (VERSÃO 3.6.0)

Pedido. Permitir que o assistente converse e entenda pedidos em linguagem livre sem o Claude, com um modelo executado no próprio computador.

Construção. Foi criado um adaptador para o Ollama (Ollama, 2026), servidor de modelos locais acessado por HTTP, escrito apenas com a biblioteca padrão. A raiz de composição passou a escolher o melhor cérebro disponível, na ordem Claude, modelo local e modo offline, com uma variável que restringe a escolha. O modelo padrão, de 3 bilhões de parâmetros, foi escolhido por responder em tempo aceitável no processador, sem placa de vídeo.

Problema e decisão. Os primeiros testes mostraram que um modelo pequeno é bom para conversar, mas pouco confiável para escolher ferramentas: um pedido de curiosidade abriu o navegador e um desabafo listou as tarefas. Por isso, a resposta passou a ser montada em camadas:

a) comandos que o modo offline reconhece são atendidos por ele, na hora e sempre do mesmo jeito;
b) pedidos de agenda em linguagem livre vão ao modelo apenas com as ferramentas do assunto e somente as seguras (criar e consultar); concluir, remover e abrir programas ficam com os comandos explícitos;
c) o restante é conversa e vai ao modelo sem ferramentas, com o texto em fluxo contínuo;
d) se o servidor local não responder, a fala é atendida pelo modo offline, e o modelo só volta a ser tentado depois de um intervalo.

Chamadas de ferramenta escritas como texto pelo modelo são executadas ou descartadas, mas nunca faladas. O histórico é compactado a cada troca, e o modelo é carregado na memória antes da primeira fala, com as mesmas opções da conversa, para evitar uma segunda carga.

Verificação. 576 testes. As camadas, as ferramentas seguras, a queda para o modo offline e o cliente HTTP são testados contra um servidor falso; no uso real, a primeira palavra de uma conversa saiu em 1 a 2,5 s, e pedidos com ferramentas levaram de 15 a 45 s no processador.

6.19 INCREMENTO 19: ESCOLHA DO MICROFONE E INICIALIZAÇÃO (VERSÃO 3.7.0)

Pedido. Permitir selecionar o microfone entre os dispositivos do computador e configurar o sistema para abrir junto com o computador ou pelo comando `robofael` no terminal.

Construção. Duas portas novas foram criadas, uma para listar os dispositivos de áudio e outra para a inicialização. Os microfones são listados pelo servidor de áudio (PipeWire (PipeWire, 2026) ou PulseAudio), sem os monitores das saídas de som, e a escolha é salva nas preferências. Abrir o dispositivo físico diretamente disputaria o aparelho com o servidor de áudio e exigiria taxa e canais exatos; por isso, o microfone continua sendo aberto pelo dispositivo padrão, e a escolha é indicada ao servidor por variáveis de ambiente definidas apenas durante a abertura, sob uma trava, para não serem herdadas pelos programas que o assistente abre. A troca vale na próxima escuta: o microfone é recalibrado e a escuta passiva é reaberta no novo dispositivo. A inicialização usa um atalho de sessão no padrão de entradas `.desktop` (Freedesktop.org, 2020), com atraso de 5 s e verificação de que o programa ainda existe, e atalhos em `~/.local/bin` para o terminal. O estado é lido dos próprios arquivos, e desligar só apaga atalhos criados para este programa. A tela de configurações passou a ter duas colunas.

Verificação. 601 testes. Uma fixture passou a apontar a pasta pessoal para uma pasta temporária, para que nenhum teste altere a configuração real do usuário. No aplicativo real, a lista do servidor de áudio confirmou que a gravação passou a sair do microfone escolhido, e não do padrão.

6.20 INCREMENTO 20: BANCO, BACKUP E MODELOS DE LINGUAGEM (VERSÃO 3.8.0)

Pedido. Permitir escolher onde o banco SQLite é salvo e fazer backup dele em qualquer local, e exibir nas configurações qual modelo de linguagem está em uso e quais modelos compatíveis estão disponíveis no Claude e no Ollama.

Construção. O banco ganhou as operações de mudar de lugar e de copiar, ambas com o assistente em uso:

a) a cópia usa a interface de backup do SQLite (SQLite, 2026), consistente mesmo com gravações em andamento, e é conferida por uma verificação de integridade antes de ser aceita;
b) ao mudar de lugar, a conexão é trocada sob a mesma trava das gravações, e o arquivo antigo é renomeado com a data, nunca apagado; a mudança é recusada se já houver um banco no destino;
c) como o banco não pode guardar o próprio endereço, o local escolhido fica em um pequeno arquivo na pasta de dados; se o banco escolhido não estiver acessível na abertura, como em um disco desconectado, o assistente avisa e usa o banco padrão apenas naquela vez, sem criar um banco vazio no lugar;

d) o backup é um arquivo único, sem o diário auxiliar, e informa quantos registros do histórico foram copiados.

Para os modelos, cada provedor ganhou um catálogo: o do Claude lista os modelos liberados para a chave, e o do Ollama lista os modelos instalados capazes de conversar e de chamar ferramentas, com o tamanho. As consultas são feitas em paralelo, fora da thread da interface, e a falta de chave ou um servidor fora do ar viram uma explicação na própria lista.

Verificação. 643 testes, incluindo a mudança do banco com gravações antes e depois, a recusa de sobrescrever outro banco, o banco ausente na abertura, a verificação da cadeia de resumos na cópia e os catálogos com respostas simuladas. No aplicativo real, a tela mostrou o modelo local em uso e os dois modelos instalados, e o backup foi conferido.

6.21 LIÇÕES APRENDIDAS NO PROCESSO

O processo evidenciou algumas lições, discutidas novamente no Capítulo 9:

a) medições com dados reais, como o volume da voz do usuário e transcrições efetivamente produzidas pelos reconhecedores, foram decisivas para resolver problemas que testes com vozes sintéticas não revelavam;
b) decisões com impacto na privacidade ou na experiência do usuário foram submetidas a ele, e configurações pessoais do sistema nunca foram alteradas sem autorização;
c) defeitos de concorrência só apareceram sob carga ou em execuções repetidas, o que justificou executar a suíte várias vezes seguidas antes de cada entrega;
d) otimizar a partir de medições, e não de suposições, levou a ganhos grandes com alterações pequenas: o custo dominante estava no redesenho da biblioteca gráfica, e não no código Python;
e) a arquitetura definida na versão 2.0.0 absorveu sem retrabalho estrutural todas as funções seguintes (ativação por voz, memória, programas, idiomas, instalador, modelo local, escolha do microfone e mudança do banco), cada uma como um adaptador novo atrás de uma porta, o que confirma o valor de investir cedo na separação de responsabilidades;
f) um modelo de linguagem pequeno precisa de limites explícitos: restringir as ferramentas que ele pode usar e deixar as ações destrutivas para comandos determinísticos tornou o modelo local útil sem expor o usuário a ações indesejadas;
g) operações sobre os dados do usuário foram desenhadas para nunca destruir informação: o arquivo antigo é renomeado, o destino ocupado é recusado e toda cópia é conferida antes de ser usada.

7 REVISÃO SEGUNDO OS PRINCÍPIOS SOLID E O CÓDIGO LIMPO

Este capítulo detalha o último incremento: a validação do projeto de ponta a ponta à luz dos princípios SOLID (Martin, 2003) e das práticas de código limpo (Martin, 2009). A revisão seguiu a disciplina de refatoração descrita por Fowler (2018): passos pequenos, comportamento preservado e suíte de testes executada após cada passo.

7.1 DIAGNÓSTICO

O diagnóstico combinou três fontes: a leitura dos módulos maiores; a análise estática com regras adicionais de complexidade (complexidade ciclomática, número de retornos, de ramificações e de comandos por função), de nomenclatura e de parâmetros sem uso; e o levantamento automático do grafo de importações entre as camadas. Os principais achados foram:

a) **SRP**: o orquestrador, com 343 linhas, acumulava o ciclo de conversa, o pipeline de fala em três estágios e a escuta passiva; o método que transformava texto em fala tinha complexidade ciclomática 20 e 57 comandos. A raiz de composição continha regras do produto (escolha de voz, idioma, ativação e saudação), e a janela principal aplicava preferências diretamente nos serviços;
b) **OCP**: o executor de ferramentas, o interpretador de intenções do modo offline e o interpretador de períodos eram cadeias de condicionais (complexidade 12, 12 e 13), que precisavam ser editadas a cada nova ferramenta, intenção ou expressão;
c) **DIP**: o núcleo da sessão dependia da classe concreta do banco SQLite; o domínio importava contratos da camada de portas (sentido inverso ao desejado); e a ferramenta de gravação de voz criava adaptadores por conta própria, sem passar pela raiz de composição;
d) **ISP**: a interface acessava o sintetizador e o reprodutor por inspeção dinâmica de atributos, sem contrato, e o gravador guardava o microfone como um objeto de tipo indeterminado;
e) **LSP**: o repositório JSON legado declarava a assinatura do contrato de persistência da agenda, mas ignorava os registros de auditoria, de modo que substituí-lo pelo SQLite perderia o histórico em silêncio;
f) **código limpo**: uma função de desenho da camisa do avatar com 57 comandos e variáveis de uma letra maiúscula; a extração do comando de voz com complexidade 12 e limiares numéricos sem nome; um diálogo de configurações com dez parâmetros; e parâmetros não usados em funções anônimas.

7.2 RESPONSABILIDADE ÚNICA

7.2.1 Divisão do orquestrador

O orquestrador foi dividido em três classes, cada uma com um único motivo para mudar:

a) `PipelineFala` (`aplicacao/pipeline.py`) transforma um fluxo de texto em fala. Cada estágio virou um método curto (pensar, sintetizar e tocar), e o retorno é um objeto imutável com o texto falado e a indicação de interrupção;
b) `VigiaAtivacao` (`aplicacao/vigia.py`) cuida exclusivamente da escuta passiva: reserva e libera o microfone sob uma trava, conta falhas seguidas e se desliga com aviso. Ela recebe por injeção duas funções: uma que informa se há interação em andamento e outra a ser chamada quando a ativação ocorrer;
c) `Orquestrador` passou a apenas coordenar: decide quando ouvir, pensar e falar, garante uma interação por vez e encaixa os anúncios entre as interações.

A interface pública do orquestrador foi preservada. Para isso, atributos como o reprodutor, o sintetizador e o detector de ativação tornaram-se propriedades que delegam aos colaboradores, e os testes existentes continuaram válidos sem alteração. O tamanho do módulo caiu de 343 para 255 linhas, e o método de fala passou de complexidade 20 para 2.

7.2.2 Sessão separada da composição e da interface

As políticas da sessão foram movidas para `aplicacao/sessao.py`: precedência do idioma, escolha da voz (preferência, perfil gravado e padrão), ativação inicial, texto da saudação, troca de idioma ao vivo e aplicação das preferências salvas. A raiz de composição ficou restrita a escolher e conectar implementações. A janela principal, por sua vez, deixou de aplicar regras: ao salvar as configurações, ela delega à sessão e apenas retraduz os componentes visuais e troca o avatar.

7.3 ABERTO PARA EXTENSÃO, FECHADO PARA MODIFICAÇÃO

Três despachos condicionais foram substituídos por estruturas extensíveis:

a) **ferramentas do modelo**: o executor mantém uma tabela que associa o nome de cada ferramenta a um método. A execução reduziu-se a validar a entrada e chamar a ação correspondente (complexidade 1). Acrescentar uma ferramenta significa declarar seu esquema e registrar um método, e um teste garante que toda ferramenta declarada tem uma ação e vice-versa;
b) **intenções do modo offline**: cada intenção (histórico, anotar, concluir, lembrar, abrir programa, listar pendentes, hora e data) é um método que devolve a resposta ou nada. O modelo percorre a cadeia em ordem e responde com a primeira intenção que se aplicar, no padrão de cadeia de responsabilidade (Gamma *et al.*, 1994). A ordem da

cadeia expressa a regra de negócio de que comandos de agenda têm prioridade sobre a abertura de programas;

c) **períodos**: cada forma de expressar um período virou uma função interpretadora pequena, que recebe um contexto imutável (texto normalizado, data de referência e idioma) e devolve o período ou nada. A lista ordenada, do mais específico para o mais genérico, substituiu a função com complexidade 13 por uma de complexidade 1.

7.4 SUBSTITUIÇÃO DE LISKOV

O banco SQLite foi verificado contra todos os contratos de persistência (agenda, histórico, preferências e o contrato combinado). O repositório JSON, que desde a versão 3.0.0 serve apenas para ler o formato legado durante a migração, não guarda o histórico de auditoria. Sua assinatura, porém, aceitava os registros e os descartava, o que o tornava um falso substituto: o verificador de tipos aceitaria usá-lo no lugar do banco, e a memória de longo prazo seria perdida em silêncio. O parâmetro foi removido. Com isso, o repositório deixou de satisfazer o contrato da agenda, e o próprio verificador de tipos passou a impedir a substituição indevida, aplicando a noção comportamental de subtipo de Liskov; Wing (1994).

7.5 SEGREGAÇÃO DE INTERFACES

Foram criadas duas portas pequenas para eliminar o acesso sem contrato:

a) `SintetizadorConfiguravel`: um sintetizador cuja voz, tom e ritmo podem ser lidos e trocados ao vivo, usado pela tela de configurações e pela troca de idioma;
b) `CapturaAudio`: a capacidade de gravar uma frase sem transcrevê-la, usada pela ferramenta de gravação da voz.

A porta `PersistenciaAssistente` foi definida como a combinação dos contratos de agenda, histórico e preferências (na versão 3.8.0, somou-se o de arquivo do banco), usada apenas pelo núcleo da sessão; os demais componentes continuam dependendo somente da parte de que precisam. A tipagem mais precisa revelou um caso real: o gravador poderia, em tese, iniciar uma gravação antes de o microfone existir, e o código passou a repassar explicitamente o microfone já verificado.

7.6 INVERSÃO DE DEPENDÊNCIA

Três ajustes alinharam todas as dependências à regra de dependência:

a) o contrato de persistência exigido pela agenda e a entidade que representa um programa instalado foram movidos para o domínio, que passou a não importar nada além do catálogo de mensagens; a camada de portas os reexporta, mantendo a compatibilidade;

b) o núcleo da sessão passou a depender da porta de persistência, e não da classe do SQLite; a leitura do perfil de voz passou a ser injetada como uma função;
c) a ferramenta de gravação de voz passou a obter o microfone e a reprodução de amostras por meio de fábricas da raiz de composição, deixando de importar adaptadores diretamente.

7.7 PRÁTICAS DE CÓDIGO LIMPO

Além das refatorações associadas aos princípios, foram aplicadas as seguintes práticas:

a) **funções pequenas**: o desenho da camisa do avatar foi decomposto em etapas com nome (sombreado, tecido, placas dos ombros, gola metálica e linhas luminosas), com um objeto de valor para a geometria da gola; a extração do comando de voz foi dividida em três estratégias nomeadas;
b) **nomes que revelam intenção**: variáveis de uma letra maiúscula foram renomeadas para largura e altura, e os limiares numéricos da extração do comando receberam constantes nomeadas;
c) **poucos argumentos**: o diálogo de configurações passou a receber dois objetos-parâmetro, um com as opções oferecidas e outro com as ações dos botões, em vez de dez argumentos;
d) **sem parâmetros mortos**: parâmetros não usados foram removidos ou marcados explicitamente;
e) **sem duplicação**: a preparação da janela de teste, repetida em dois arquivos de teste, foi unificada.

Para garantir que o comportamento visual não mudasse, o resumo SHA-256 de todos os 72 quadros dos dois avatares foi calculado antes e depois da decomposição da função de desenho, e os resumos coincidiram.

7.8 REGRAS PERMANENTES

Para que a qualidade obtida não se perdesse nas evoluções seguintes, as regras passaram a ser verificadas automaticamente em toda execução de `make verificar` e na integração contínua:

a) complexidade ciclomática máxima de 10 por função (McCabe, 1976);
b) limites de retornos, ramificações e comandos por função;
c) convenções de nomenclatura e ausência de parâmetros sem uso;
d) verificação estática de tipos sem ocorrências;
e) cobertura mínima de 80% das linhas e ramificações;
f) testes de arquitetura, descritos na Seção 8.4.

As exceções às regras foram documentadas na própria configuração com sua justificativa. Uma delas permite nomes de exceção do domínio em português, como `TarefaNaoEncontrada`,

sem o sufixo `Error`. Outra libera os dublês de teste de usar todos os argumentos dos contratos que imitam.

7.9 MÉTRICAS ANTES E DEPOIS

A Tabela 2 compara as funções e os módulos alterados.

Tabela 2 – Métricas antes e depois da revisão

Elemento (métrica)	**Antes**	**Depois**
Orquestrador (linhas do módulo)	343	255
Método de fala do orquestrador (complexidade)	20	2
Execução de ferramentas (complexidade)	12	1
Interpretação offline (complexidade)	12	3
Interpretação de períodos (complexidade)	13	1
Extração do comando de voz (complexidade)	12	5
Desenho da camisa (comandos na função)	57	22
Diálogo de configurações (parâmetros)	10	5
Maior complexidade ciclomática do projeto	20	10
Ocorrências das novas regras (complexidade, tamanho, nomes e parâmetros)	22	0
Dependências entre camadas fora da regra	2	0
Testes automatizados	508	529

Fonte: elaboração própria (2026), com dados do ruff e do pytest.

A redução da complexidade não foi obtida à custa de mais código total: os colaboradores extraídos somam linhas, mas cada função passou a ser curta, nomeada e testável isoladamente, o que é o objetivo das práticas de código limpo.

8 TESTES E VALIDAÇÃO

Este capítulo apresenta a estratégia de testes, as suítes que compõem a validação automatizada, os testes de arquitetura que verificam os princípios de projeto, o processo de integração contínua e os procedimentos de validação manual.

8.1 ESTRATÉGIA

A estratégia segue a pirâmide de testes (Cohn, 2009). A base é formada por testes unitários rápidos das regras de domínio, das políticas de aplicação e do processamento de sinais. O nível intermediário reúne testes de integração dos adaptadores com dublês das bibliotecas externas e do banco SQLite real em arquivo temporário. No topo ficam poucos testes que abrem a janela Tk real, marcados como lentos e executados com servidor gráfico virtual na integração contínua.

Três decisões de projeto tornaram essa estratégia viável. A primeira é a arquitetura hexagonal: todo serviço externo (modelo de linguagem, reconhecedores, sintetizador, reprodutor, lançador de programas) é acessado por uma porta e pode ser substituído por um dublê (Meszaros, 2007). A segunda é a injeção do relógio, que torna determinísticos os testes de lembretes e de períodos. A terceira é a reprodução simulada com duração configurável, que permite provar propriedades temporais do pipeline, como a de que a primeira frase começa a tocar antes de o modelo terminar a resposta.

8.2 DUBLÊS DE TESTE

O módulo comum de testes oferece dublês para cada porta:

a) modelos roteirizados, que devolvem pedaços de texto com atraso controlado;
b) um cliente falso do modelo Claude, que reproduz o protocolo de transmissão em fluxo contínuo com blocos de texto e de ferramenta;
c) sintetizadores que registram os textos e podem falhar sob demanda;
d) reprodutores que registram as faixas tocadas e acionam ganchos durante a reprodução;
e) reconhecedores com falas roteirizadas e transcrições opcionais;
f) detectores de ativação que devolvem ativações ou falhas roteirizadas e depois aguardam o cancelamento;
g) lançadores de programas vazios ou com listas controladas;
h) um servidor HTTP falso do Ollama e catálogos de modelos roteirizados, que respondem com listas ou com falhas (sem chave, sem conexão, servidor fora do ar);
i) listas de microfones e estados de inicialização controlados, sem depender do hardware.

Uma fixture automática aponta a pasta pessoal e a pasta de configuração para diretórios temporários em todos os testes, de modo que nenhum deles possa criar atalhos de inicialização ou comandos na pasta real do usuário. Também são oferecidos geradores de áudio sintético, como senoides em frequências conhecidas e arquivos WAV temporários, para validar a análise de tom e de envelope.

8.3 SUÍTES DE TESTE

O Quadro 12 descreve as suítes e o número de casos de teste de cada uma, contando as variações parametrizadas.

Quadro 12 – Suítes de teste automatizado

Suíte	**Casos**	**O que garante**
`test_i18n`	190	Catálogo completo e com os mesmos campos nas duas línguas; comandos, períodos e respostas em inglês; troca ao vivo; precedência da configuração
`test_ativacao`	79	Gramática de “ok Fael” e busca fonética com saídas reais do reconhecedor local; frases parecidas que não podem disparar; extração do comando com saídas reais do reconhecedor remoto; vigia liberando o microfone; desligamento após falhas
`test_memoria`	41	Dezenove expressões de período, intenção “o que eu fiz”, resultado líquido e respostas
`test_programas`	29	Leitura de entradas `.desktop`, extração do pedido, ranqueamento, sinônimos, ambiguidade, abertura sem interpretador de comandos e prioridade da agenda
`test_ollama`	24	Respostas em camadas, ferramentas seguras, chamadas escritas como texto, queda para o modo offline, interrupção, histórico, idioma, cliente HTTP contra servidor falso e escolha do cérebro
`test_sqlite`	18	Esquema versionado, transação conjunta de estado e histórico, gatilhos de somente acréscimo, detecção de quatro tipos de adulteração, consulta por período, concorrência, migração e preferências
`test_agenda`	18	Regras da agenda e transações (persiste e notifica somente em caso de sucesso)
`test_analise_audio`	23	Estimativa de tom com erro de até 2%, envelope de volume e timbre e corte do silêncio das pontas (sem cortar consoantes fracas)
`test_orquestrador`	17	Prova do pipeline, interrupção, modo contínuo, anúncios em fila, falhas de modelo, síntese e microfone, corrida do cancelamento e pedidos digitados durante a fala ou a escuta
`test_modelo_offline`	15	Comandos, plural, lembretes relativos e despedida
`test_animacao`	15	Suavização independente da taxa de quadros e seleção do quadro da boca

Suíte	Casos	O que garante
`test_modelo_claude`	14	Transmissão real do texto, laço de ferramentas, limite de tamanho, recusa, erros da API, descarte da rodada e limite do histórico
`test_arquitetura`	15	Regra de dependência entre camadas e princípios SOLID (Seção 8.4)
`test_configuracoes`	15	Conversões de tom e ritmo, precedência das preferências, janela de tamanho fixo e diálogo de configurações, inclusive microfone, inicialização, modelo em uso, mudança e backup do banco
`test_microfone_inicializacao`	14	Lista de microfones sem monitores, variável de escolha restrita à abertura, rótulos, reabertura da escuta passiva, atalho de sessão e comandos no terminal sem apagar arquivos de outra origem
`test_banco_e_modelos`	13	Backup conferido, mudança do banco com gravações antes e depois, recusa de sobrescrever, banco ausente na abertura, catálogos do Claude e do Ollama e texto da lista de modelos
`test_perfil_voz`	12	Criação, leitura e tolerância a arquivos inexistentes ou corrompidos
`test_ferramentas`	9	Esquemas e validação de entradas (tipos, campos extras, booleano diferente de inteiro)
`test_avatar`	9	Busto centralizado, camisa, tamanho uniforme dos quadros e cache
`test_vad`, `test_estados_barramento`, `test_frases`	23	Detecção de fala, transições de estado, entrega de eventos entre threads e divisão de frases
`test_sessao_pipeline_vigia`	8	Pipeline e vigia isolados, síntese paralela com ordem preservada e aplicação das preferências
`test_renderizacao`	8	Canvas diferencial, fila de trabalho com orçamento, laço de quadros adaptativo, halos opacos e regressão de desempenho do palco em repouso
`test_ganho`, `test_configuracao_tema`, `test_adaptadores_voz`	21	Ganho automático, configuração (inclusive os nomes antigos das variáveis) e tema, cache LRU de voz e reprodutor com mixer real
`test_agendador`, `test_repositorio_json`, `test_bootstrap_ui`	13	Lembrete no horário exato, formato legado, composição, queda para o modo offline e janela Tk real
Total	**643**	

Fonte: elaboração própria (2026), a partir da coleta do pytest.

8.4 TESTES DE ARQUITETURA

Inspirados nas funções de aptidão arquitetural de Ford; Parsons; Kua (2017), os testes de arquitetura transformam as regras de projeto em verificações executáveis. Eles estão no arquivo `tests/test_arquitetura.py` e verificam:

a) **regra de dependência**: a árvore sintática de cada módulo é analisada, todas as

importações internas são extraídas e cada uma é comparada com uma tabela que declara, para cada camada, as camadas que ela pode importar; qualquer violação é relatada com arquivo, linha e sentido da dependência;

b) **cobertura das regras**: toda camada existente no pacote precisa ter uma regra declarada, para que uma camada nova não escape da verificação;
c) **pureza do núcleo**: o domínio e as portas não podem importar bibliotecas de infraestrutura (banco de dados, interface gráfica, processamento numérico, áudio, rede, modelo de linguagem nem execução de processos);
d) **LSP**: o banco SQLite precisa oferecer todos os métodos de todos os contratos de persistência, e o repositório legado não pode ter a assinatura que promete guardar o histórico;
e) **DIP**: a agenda precisa funcionar com um repositório em memória escrito no próprio teste, sem banco de dados;
f) **OCP**: toda ferramenta declarada ao modelo precisa ter uma ação registrada e vice-versa, e uma intenção nova acrescentada à cadeia do modo offline precisa funcionar sem alterar o código existente;
g) **SRP**: os métodos do pipeline e da vigia não podem voltar ao orquestrador, e as políticas da sessão não podem voltar à raiz de composição.

8.5 ANÁLISE ESTÁTICA E INTEGRAÇÃO CONTÍNUA

O comando `make verificar` executa, em sequência, a análise estática (ruff), a verificação de tipos (mypy) e a suíte completa (pytest). O mesmo conjunto é executado pelos ganchos de *pre-commit* e pela integração contínua no GitHub Actions, em Python 3.10 e 3.12, com o servidor gráfico virtual Xvfb para os testes de interface. A medição de cobertura falha se o total ficar abaixo de 80%.

8.6 COBERTURA

A cobertura combinada de linhas e ramificações foi de 86,6% sobre 4.694 comandos. A Tabela 3 apresenta a cobertura por camada.

Tabela 3 – Cobertura de testes por camada (linhas e ramificações)

Camada	**Comandos**	**Cobertura (%)**
Portas	85	100,0
Internacionalização	68	98,7
Domínio	551	98,2
Aplicação	758	95,5
Avatar	290	95,2
Configuração	117	93,8

Camada	Comandos	Cobertura (%)
Áudio	260	90,8
Adaptadores	1.297	82,1
Apresentação	1.130	76,0
Raiz de composição	136	75,0
Total	**4.694**	**86,6**

Fonte: elaboração própria (2026), com dados do pytest-cov.

A distribuição é coerente com a arquitetura: as camadas internas, que concentram as regras, têm cobertura próxima de 100%. As menores coberturas estão nos adaptadores de hardware (microfone, áudio nativo e escuta local), cuja execução exige dispositivos reais, e na interface gráfica, cujos caminhos de desenho são parcialmente exercitados pelos testes com a janela real.

8.7 VALIDAÇÃO MANUAL

Os aspectos que dependem de hardware ou de percepção humana foram validados manualmente:

a) a aparência do avatar, por meio de prévias geradas com as mesmas imagens e posições da interface;
b) o reconhecimento da frase de ativação com a voz real do usuário, acompanhado pelos registros detalhados, que mostram o texto reconhecido, o volume de pico e o ganho aplicado;
c) a gravação real da voz e a criação do perfil;
d) a migração dos dados reais, feita primeiro em uma cópia;
e) a execução em inglês;
f) a abertura da janela a partir da instalação simulada do pacote, com a verificação da classe da janela e da criação das pastas de dados e de cache do usuário;
g) a escolha do microfone, conferida na lista de gravações ativas do servidor de áudio, que mostrou o assistente gravando do microfone escolhido, e não do padrão;
h) a tela de configurações com os serviços reais e uma pasta de dados temporária, mostrando o modelo local em uso, os modelos instalados e um backup conferido;
i) o atalho de sessão gerado, validado pela ferramenta `desktop-file-validate`.

8.8 ESTABILIDADE DA SUÍTE

Como o sistema é concorrente, uma suíte aprovada uma única vez não basta. Antes de cada entrega, a suíte completa foi executada repetidamente, inclusive com a máquina sob carga. Esse procedimento revelou e permitiu corrigir quatro defeitos intermitentes ao longo do projeto:

dois testes que dependiam de tempo sob carga, a liberação de variáveis da interface fora da thread principal e o bloqueio da thread de renderização pelo coletor de lixo, descrito no Capítulo 6.

9 RESULTADOS E DISCUSSÃO

Este capítulo apresenta os resultados obtidos em relação aos requisitos e aos objetivos, discute as decisões de projeto à luz desses resultados e expõe as limitações identificadas.

9.1 ATENDIMENTO AOS REQUISITOS FUNCIONAIS

Os 42 requisitos funcionais especificados no Capítulo 4 foram implementados. Destes, 41 são verificados por testes automatizados. O requisito de instalação (RF36) é verificado por um roteiro próprio, que instala o pacote em uma pasta temporária e confere seu funcionamento sem alterar o sistema. Algumas funções dependem de serviços externos ou de hardware, e sua verificação automatizada usa dublês: a conversa livre com o modelo de linguagem (RF24), o reconhecimento remoto (RF03 e RF05) e a voz neural (RF06). A escolha do microfone (RF39) foi conferida também no servidor de áudio real. Nessas funções, a validação em uso real complementou os testes. A rastreabilidade completa está no Apêndice A.

9.2 ATENDIMENTO AOS REQUISITOS NÃO FUNCIONAIS

A Tabela 4 compara as medições de desempenho da arquitetura inicial, serial, com as da arquitetura em pipeline introduzida na versão 2.0.0 e mantida até a versão final.

Tabela 4 – Medições de desempenho

Medida	**Arquitetura inicial**	**Arquitetura final**
Janela visível após abrir	0,5 a 4 s	0,8 s
Primeira palavra (modo offline, texto)	resposta e voz inteiras	cerca de 1,3 s
Primeira palavra (modelo de linguagem)	geração e síntese completas	primeira frase gerada e sintetizada
Tempo para interromper	apenas durante a reprodução	cerca de 10 ms, em qualquer estágio
Renderizar o avatar sem cache (crachá / *selfie*)	2,9 s / 4,3 s	1,8 s / 2,2 s, em segundo plano
Ler o avatar do cache	cerca de 0,4 s	0,08 s / 0,2 s
Repetir uma fala já sintetizada	nova síntese	0,1 ms (cache LRU)
Atraso de um lembrete	até 15 s	menos de 0,1 s
Pausa de fim de fala	0,9 s fixos	0,7 s, ignorando ruídos curtos

Fonte: elaboração própria (2026).

O Incremento 16 (versão 3.4.0) acrescentou um refinamento guiado por medições do aplicativo real, apresentado na Tabela 5. O consumo em repouso foi medido em uma janela de

20 a 30 s depois do fim da saudação falada, com as duas versões nas mesmas condições e com o cache do avatar já pronto.

Tabela 5 – Medições de desempenho e fluidez: versões 3.3.0 e 3.4.0

Medida	**Versão 3.3.0**	**Versão 3.4.0**
Processamento em repouso	29% de um núcleo	5% de um núcleo
Custo de um quadro do palco (microbenchmark)	13 ms	5 ms
Janela visível após abrir	1,7 a 2,4 s	0,6 a 0,7 s
Primeira palavra da saudação	3,1 a 3,2 s	2,4 a 2,8 s
Duração da saudação	cerca de 10,1 s	cerca de 8,5 s
Silêncio entre frases faladas	cerca de 1,2 s	0,15 s
Engasgo ao trocar de avatar	cerca de 170 ms	nenhum (no máximo 4 ms por quadro)
Memória residente	cerca de 369 MB	cerca de 368 MB

Fonte: elaboração própria (2026).

A memória não diminuiu: cerca de 150 MB correspondem às bibliotecas (o SDK do modelo de linguagem, o reconhecedor local com seu modelo, o mixer de áudio e o processamento numérico) e cerca de 100 MB às imagens já convertidas para a biblioteca gráfica. Um teste com a devolução explícita de memória ao sistema operacional recuperou apenas 10 MB, o que indicou uso legítimo, e não fragmentação.

Os requisitos de desempenho (RNF01 a RNF05, RNF26 e RNF27) foram atendidos conforme as tabelas anteriores. Os requisitos de confiabilidade (RNF06 a RNF09) foram atendidos e são verificados por testes que simulam falhas de transação, adulterações do histórico, interrupções durante o fluxo de texto, recusas e erros da API, ausência de microfone e falhas repetidas da escuta passiva. O requisito de exatidão da ativação (RNF13) foi atendido no conjunto de calibração, com zero falsos positivos em 115 frases parecidas, e confirmado em uso real com a voz do usuário depois da introdução do ganho automático. Os requisitos de manutenibilidade (RNF19 a RNF22) foram atendidos e passaram a ser verificados continuamente: nenhuma dependência fora da regra entre camadas, complexidade máxima de 10, tipos verificados sem ocorrências, com todas as assinaturas tipadas, e cobertura de 85,9%, acima do mínimo de 80%.

9.3 DISCUSSÃO

9.3.1 Latência percebida

O maior ganho de experiência veio de uma decisão arquitetural, e não de otimizações locais. Ao sobrepor a geração de texto, a síntese e a reprodução, o tempo até a primeira palavra deixou de depender do tamanho da resposta. Esse resultado está de acordo com a observação

de Nielsen (1993) de que respostas em torno de um segundo preservam o fluxo da interação. O custo dessa escolha é a concorrência, que exigiu mecanismos explícitos: token de cancelamento por interação, máquina de estados e barramento de eventos.

9.3.2 Privacidade e exatidão da ativação

O uso de um reconhecedor local com gramática restrita mostrou-se superior à transcrição livre para uma frase de ativação curta e fora do vocabulário. As frases distratoras funcionam como uma classe de rejeição explícita, o que explica a ausência de falsos positivos na calibração. Os testes com a voz real, por outro lado, mostraram que a qualidade da captação é tão importante quanto o algoritmo: com o microfone a 37% de entrada, nenhum ajuste de gramática resolveria o problema. O ganho automático melhorou a situação, mas não substitui um volume de entrada adequado.

9.3.3 Memória auditável

O encadeamento de resumos criptográficos, somado aos gatilhos de somente acréscimo, oferece duas camadas de proteção: o próprio banco recusa alterações feitas pelo sistema e por ferramentas comuns, e a verificação da cadeia detecta modificações feitas por fora, inclusive com o arquivo aberto em outro programa. Como os registros guardam a descrição no momento do fato, a pergunta “o que eu fiz” é respondida corretamente mesmo depois da remoção dos itens.

9.3.4 Princípios SOLID na prática

A revisão final mostrou que uma arquitetura em camadas bem definida não impede, por si só, a erosão local da qualidade. Mesmo com a separação hexagonal desde a versão 2.0.0, as funções acumularam condicionais, o orquestrador absorveu responsabilidades e duas dependências passaram a apontar no sentido errado ao longo de nove incrementos. Os princípios SOLID funcionaram como critérios objetivos para localizar e corrigir esses pontos, e as funções de aptidão de Ford; Parsons; Kua (2017) tornaram a correção permanente. Em vez de depender da atenção de quem altera o código, a violação passa a ser apontada pelo próprio processo de verificação.

Um efeito colateral positivo foi a melhoria da tipagem. Ao substituir acessos dinâmicos por contratos explícitos, o verificador de tipos encontrou um caso real de uso possivelmente nulo, e a correção do princípio de Liskov transformou um risco silencioso de perda de dados em um erro de tipo.

9.3.5 Desenvolvimento assistido por inteligência artificial

A experiência indica que um agente de programação acelera substancialmente a implementação, inclusive de refatorações amplas, mas desloca a responsabilidade pela qualidade para as verificações automáticas e para a validação com o usuário. Os defeitos mais difíceis do projeto foram identificados por medições reais ou por execuções repetidas, como o volume baixo do microfone, a configuração que redirecionava a API e o bloqueio da thread pelo coletor de lixo, e não pela leitura do código. Isso reforça o valor das práticas de engenharia descritas neste trabalho, que se mostraram complementares ao uso do agente.

9.4 LIMITAÇÕES

As principais limitações identificadas são:

a) o reconhecimento de comandos e a voz neural dependem de serviços na internet; sem conexão, a conversa livre pode usar o modelo local, mas a voz de reserva também fica indisponível;
b) o modelo local executado no processador leva de 15 a 45 s em pedidos que usam ferramentas e, por ser pequeno, só recebe ferramentas que criam ou consultam: concluir, remover e abrir programas exigem os comandos explícitos;
c) a tela de configurações mostra os modelos disponíveis, mas a troca do modelo ainda é feita por variáveis de ambiente e vale ao reabrir o assistente;
d) a restauração de um backup é manual, com o assistente fechado;
e) a gramática restrita foi calibrada apenas para a frase “ok Fael”; outras frases usam a busca fonética livre, menos precisa;
f) a calibração da ativação usou cinco vozes sintéticas e a voz de um único usuário real; uma avaliação com mais pessoas e ambientes ruidosos daria estimativas mais robustas de exatidão;
g) os lembretes só são anunciados com o programa aberto; um lembrete vencido com o programa fechado é anunciado na abertura seguinte;
h) o pacote de instalação foi construído e testado para Ubuntu 24.04 com Python 3.12; outras versões exigem reconstruí-lo na própria máquina, e as fotografias pessoais seguem dentro do pacote;
i) as posições do rosto em cada fotografia foram medidas manualmente; um novo avatar exige cadastrar essas posições no catálogo.

10 CONCLUSÃO

Este trabalho apresentou o Robofael, um assistente pessoal de atividades diárias para Linux que conversa por voz, é representado por um avatar animado criado a partir das fotografias do usuário, organiza tarefas e lembretes, mantém uma memória de longo prazo auditável, abre programas instalados, funciona em português e em inglês e pode ser instalado no Ubuntu por um pacote padrão. Sem o Claude, um modelo de linguagem local assume a conversa; e a tela de configurações permite escolher o microfone, abrir o assistente com o computador, mudar o banco de lugar e fazer backups conferidos. O problema de pesquisa, que buscava construir um assistente responsivo, respeitoso com a privacidade, com histórico confiável e estrutura interna fácil de evoluir, foi respondido com uma solução que combina decisões arquiteturais e verificações automáticas.

Os objetivos específicos foram atingidos. O avatar em formato de busto, com boca sincronizada em 36 combinações de abertura e formato, foi gerado a partir das fotografias. O ciclo de conversa em pipeline reduziu a espera à primeira frase da resposta. A ativação pela frase “ok Fael” é processada localmente, com gramática restrita e zero falsos positivos na calibração. A memória de longo prazo em SQLite é somente de acréscimo e protegida por cadeia de resumos SHA-256, e responde perguntas sobre dias, semanas e outros períodos. A integração com o ambiente Linux abre programas por nome ou sinônimo. A internacionalização permite trocar o idioma ao vivo. O instalador para Ubuntu foi testado em ambiente isolado. Por fim, a revisão segundo os princípios SOLID e as práticas de código limpo foi acompanhada de testes de arquitetura que tornam essas regras permanentes. Os requisitos funcionais e não funcionais foram especificados, rastreados até os testes e documentados, assim como todo o processo de construção.

Como contribuição, além do próprio artefato de software, o trabalho mostra de forma concreta como os princípios SOLID se materializam em um sistema concorrente, com integração a serviços externos e interface gráfica: classes com uma única responsabilidade no lugar de um coordenador sobrecarregado; tabelas extensíveis no lugar de cadeias de condicionais; contratos pequenos e pertencentes à camada que deles precisa; e um caso real em que respeitar o princípio de Liskov transformou uma perda silenciosa de dados em um erro detectado pelo verificador de tipos. O trabalho também mostra que testes de arquitetura, limites de complexidade e cobertura mínima, verificados continuamente, preservam a qualidade estrutural ao longo da evolução, inclusive quando a implementação é acelerada por um agente de inteligência artificial. O Incremento 16 ilustra o valor de otimizar a partir de medições: com uma camada que só repassa à biblioteca gráfica o que mudou, imagens opacas e uma taxa de quadros adaptativa, o consumo em repouso caiu de 29% para 5% de um núcleo, sem alterar a aparência, e um teste determinístico passou a impedir que esse ganho se perca. Por fim, os quatro últimos incrementos mostram que a arquitetura absorveu um novo cérebro, novos dispositivos e novas operações sobre os dados

apenas com portas e adaptadores novos, e que um modelo pequeno se torna útil quando o sistema limita o que ele pode fazer.

10.1 TRABALHOS FUTUROS

A partir das limitações identificadas, sugerem-se as seguintes evoluções:

a) adotar reconhecimento de comandos e síntese de voz locais, que, somados ao modelo de linguagem local já existente, eliminariam a dependência de internet para as funções principais;
b) permitir escolher o modelo de linguagem na própria tela de configurações, com troca ao vivo;
c) restaurar backups pela interface e criar backups automáticos periódicos;
d) gerar automaticamente a gramática restrita de qualquer frase de ativação, com frases distratoras obtidas por semelhança fonética;
e) avaliar a exatidão da ativação com um grupo maior de pessoas, sotaques e níveis de ruído;
f) detectar automaticamente os pontos do rosto em novas fotografias, dispensando o cadastro manual;
g) executar o agendador como serviço do sistema, para anunciar lembretes mesmo com a janela fechada;
h) sincronizar a memória entre computadores com criptografia de ponta a ponta, mantendo a cadeia de integridade;
i) publicar o pacote em um repositório próprio, com assinatura, para atualizações automáticas pelo gerenciador de pacotes;
j) adicionar novos idiomas, aproveitando o catálogo e o teste de completude das traduções.

REFERÊNCIAS

ALPHA CEPHEI. **Vosk speech recognition toolkit**. [*S. l.*]: Alpha Cephei, 2026. Disponível em: https://alphacephei.com/vosk/. Acesso em: 22 out. 2026.

ANTHROPIC. **Claude API documentation**. San Francisco: Anthropic, 2026. Disponível em: https://docs.anthropic.com/. Acesso em: 22 out. 2026.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação: referências: elaboração. Rio de Janeiro: ABNT, 2018.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação: citações em documentos: apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação: trabalhos acadêmicos: apresentação. Rio de Janeiro: ABNT, 2011.

BECK, Kent. **Test-driven development**: by example. Boston: Addison-Wesley, 2003.

COCKBURN, Alistair. **Hexagonal architecture**. [*S. l.*], 2005. Disponível em: https://alistair.cockburn.us/hexagonal-architecture/. Acesso em: 22 out. 2026.

COHN, Mike. **Succeeding with agile**: software development using Scrum. Upper Saddle River: Addison-Wesley, 2009.

DEBIAN. **Debian policy manual**. [*S. l.*]: The Debian Project, 2026. Disponível em: https://www.debian.org/doc/debian-policy/. Acesso em: 22 out. 2026.

EVANS, Eric. **Domain-driven design**: tackling complexity in the heart of software. Boston: Addison-Wesley, 2004.

FORD, Neal; PARSONS, Rebecca; KUA, Patrick. **Building evolutionary architectures**: support constant change. Sebastopol: O'Reilly Media, 2017.

FOWLER, Martin. **Inversion of control containers and the dependency injection pattern**. [*S. l.*], 2004. Disponível em: https://martinfowler.com/articles/injection.html. Acesso em: 22 out. 2026.

FOWLER, Martin. **Refactoring**: improving the design of existing code. 2. ed. Boston: Addison-Wesley, 2018.

FREEDESKTOP.ORG. **Desktop entry specification**: version 1.5. [*S. l.*], 2020. Disponível em: https://specifications.freedesktop.org/desktop-entry-spec/latest/. Acesso em: 22 out. 2026.

GAMMA, Erich *et al.* **Design patterns**: elements of reusable object-oriented software. Reading: Addison-Wesley, 1994.

HARRIS, Charles R. *et al.* Array programming with NumPy. **Nature**, London, v. 585, p.

357-362, 2020.

HUNT, Andrew; THOMAS, David. **The pragmatic programmer**: from journeyman to master. Reading: Addison-Wesley, 1999.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2011**: systems and software engineering: systems and software quality requirements and evaluation (SQuaRE): system and software quality models. Geneva: ISO, 2011.

JURAFSKY, Daniel; MARTIN, James H. **Speech and language processing**: an introduction to natural language processing, computational linguistics, and speech recognition. 2. ed. Upper Saddle River: Prentice Hall, 2009.

LARMAN, Craig; BASILI, Victor R. Iterative and incremental development: a brief history. **Computer**, Los Alamitos, v. 36, n. 6, p. 47-56, 2003.

LEVKIVSKYI, Ivan; LEHTOSALO, Jukka; LANGA, Łukasz. **PEP 544**: protocols: structural subtyping (static duck typing). [*S. l.*]: Python Software Foundation, 2017. Disponível em: https://peps.python.org/pep-0544/. Acesso em: 22 out. 2026.

LISKOV, Barbara H.; WING, Jeannette M. A behavioral notion of subtyping. **ACM Transactions on Programming Languages and Systems**, New York, v. 16, n. 6, p. 1811-1841, 1994.

MARTIN, Robert C. **Agile software development**: principles, patterns, and practices. Upper Saddle River: Prentice Hall, 2003.

MARTIN, Robert C. **Código limpo**: habilidades práticas do agile software. Rio de Janeiro: Alta Books, 2009.

MARTIN, Robert C. **Arquitetura limpa**: o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2019.

McCABE, Thomas J. A complexity measure. **IEEE Transactions on Software Engineering**, New York, v. SE-2, n. 4, p. 308-320, 1976.

MESZAROS, Gerard. **xUnit test patterns**: refactoring test code. Upper Saddle River: Addison-Wesley, 2007.

MEYER, Bertrand. **Object-oriented software construction**. Englewood Cliffs: Prentice Hall, 1988.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **FIPS PUB 180-4**: secure hash standard (SHS). Gaithersburg: NIST, 2015.

NIELSEN, Jakob. **Usability engineering**. Boston: Academic Press, 1993.

OLLAMA. **Ollama API**. [*S. l.*], 2026. Disponível em: https://github.com/ollama/ollama/blob/main/docs/api.md. Acesso em: 26 out. 2026.

PIPEWIRE. **PipeWire documentation**. [*S. l.*], 2026. Disponível em:

https://docs.pipewire.org/. Acesso em: 26 out. 2026.

POVEY, Daniel *et al.* The Kaldi speech recognition toolkit. *In*: IEEE WORKSHOP ON AUTOMATIC SPEECH RECOGNITION AND UNDERSTANDING, 2011, Waikoloa. **Proceedings** [...]. [*S. l.*]: IEEE Signal Processing Society, 2011.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

PRESTON-WERNER, Tom. **Semantic versioning 2.0.0**. [*S. l.*], 2013. Disponível em: https://semver.org/. Acesso em: 22 out. 2026.

SOHN, Jongseo; KIM, Nam Soo; SUNG, Wonyong. A statistical model-based voice activity detection. **IEEE Signal Processing Letters**, New York, v. 6, n. 1, p. 1-3, 1999.

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

SQLITE. **SQLite documentation**. [*S. l.*], 2026. Disponível em: https://www.sqlite.org/docs.html. Acesso em: 22 out. 2026.

VASWANI, Ashish *et al.* Attention is all you need. *In*: CONFERENCE ON NEURAL INFORMATION PROCESSING SYSTEMS, 31., 2017, Long Beach. **Advances in neural information processing systems 30**. Red Hook: Curran Associates, 2017. p. 5998-6008.

GLOSSÁRIO

Adaptador: componente que implementa uma porta usando uma tecnologia concreta, como um banco de dados ou um serviço de voz.

Avatar: representação gráfica do assistente; neste trabalho, um busto gerado a partir de fotografias do usuário.

Cache LRU: armazenamento temporário que, quando cheio, descarta primeiro os itens usados há mais tempo.

Cadeia de resumos: sequência de registros em que cada um guarda o resumo criptográfico do anterior, tornando detectável qualquer alteração.

Dublê de teste: objeto que substitui um colaborador real durante um teste, como um sintetizador que apenas registra os textos recebidos.

Fluxo contínuo (*streaming*): entrega de um resultado em partes, à medida que é produzido.

Frase de ativação: expressão que chama o assistente sem uso das mãos; neste trabalho, “ok Fael”.

Frase distratora: frase parecida com a de ativação, incluída na gramática para absorver o que não deve ativar o assistente.

Função de aptidão arquitetural: verificação objetiva, preferencialmente automatizada, de uma característica desejada da arquitetura.

Gramática restrita: lista fechada de frases entre as quais o reconhecedor de fala deve escolher.

Pipeline: encadeamento de estágios que processam dados em sequência, de forma sobreposta no tempo.

Porta: contrato que descreve o que a aplicação precisa do mundo externo, ou o que oferece a ele.

Raiz de composição: ponto único do programa em que as implementações concretas são criadas e conectadas.

Token de cancelamento: sinal compartilhado pelos estágios de uma interação, cuja ativação interrompe todos eles.

Wheel: formato de distribuição de pacotes Python já construídos, instaláveis sem compilação.

APÊNDICE A – RASTREABILIDADE ENTRE REQUISITOS E TESTES

Este apêndice associa cada requisito à suíte que o verifica. Os nomes das suítes correspondem aos arquivos da pasta `tests`.

Quadro A.1 – Rastreabilidade dos requisitos funcionais

Requisitos	**Suítes de teste**
RF01, RF14	`test_avatar`, `test_configuracoes`
RF02	`test_animacao`, `test_analise_audio`
RF03, RF09, RF13	`test_orquestrador`, `test_vad`
RF04, RF05	`test_ativacao`, `test_ganho`, `test_sessao_pipeline_vigia`
RF06	`test_adaptadores_voz`, `test_perfil_voz`, `test_configuracoes`
RF07, RF08, RF10, RF12	`test_bootstrap_ui`, `test_orquestrador`, `test_estados_barramento`, `test_configuracoes`
RF11	`test_bootstrap_ui`, `test_i18n`
RF15 a RF17, RF19	`test_agenda`, `test_modelo_offline`, `test_ferramentas`
RF18	`test_agendador`, `test_modelo_offline`
RF20, RF22, RF23	`test_sqlite`, `test_repositorio_json`
RF21	`test_memoria`, `test_i18n`
RF24, RF26	`test_modelo_claude`, `test_ferramentas`
RF25, RF27	`test_modelo_offline`, `test_bootstrap_ui`
RF28, RF29	`test_programas`
RF30	`test_perfil_voz`, `test_analise_audio`
RF31, RF32	`test_configuracoes`, `test_sessao_pipeline_vigia`
RF33, RF34	`test_i18n`
RF35	`test_configuracao_tema`, `test_i18n`
RF36	roteiro `empacotamento/testar_deb.sh` (`make testar-deb`)
RF37	`test_ollama`
RF38	`test_banco_e_modelos`, `test_configuracoes`
RF39, RF40	`test_microfone_inicializacao`, `test_configuracoes`
RF41, RF42	`test_banco_e_modelos`, `test_configuracoes`

Fonte: elaboração própria (2026).

Quadro A.2 – Rastreabilidade dos requisitos não funcionais

Requisitos	**Verificação**
RNF01	`test_bootstrap_ui` e medição de tempo
RNF02, RNF03	`test_orquestrador`, `test_sessao_pipeline_vigia`
RNF04	`test_agendador`
RNF05	`test_animacao`, `test_avatar`, `test_adaptadores_voz`

Requisitos	**Verificação**
RNF06	`test_sqlite`
RNF07	`test_modelo_claude`
RNF08	`test_bootstrap_ui`, `test_ativacao`, `test_adaptadores_voz`
RNF09	`test_orquestrador`, `test_estados_barramento`
RNF10, RNF12, RNF17	revisão de código e do catálogo de mensagens
RNF11	`test_ferramentas`, `test_programas`
RNF13	`test_ativacao`
RNF14 a RNF16	`test_configuracoes`, `test_bootstrap_ui`
RNF18	`test_i18n`
RNF19, RNF22	`test_arquitetura`
RNF20, RNF21	ruff, mypy e pytest-cov em `make verificar` e na integração contínua
RNF23, RNF24	integração contínua (Python 3.10 e 3.12) e `make testar-deb`
RNF25	revisão do arquivo `CHANGELOG.md`
RNF26, RNF27	`test_renderizacao`, `test_analise_audio`, `test_adaptadores_voz` e medição no app real
RNF28	`test_banco_e_modelos` e a fixture `casa_isolada` (`tests/conftest.py`)

Fonte: elaboração própria (2026).

APÊNDICE B – COMANDOS DE VOZ

O Quadro B.1 apresenta exemplos de comandos aceitos. Os comandos são entendidos nos dois idiomas, independentemente do idioma das respostas. Com o modelo de linguagem ativo, qualquer formulação em linguagem natural é aceita; os exemplos abaixo também funcionam no modo offline, exceto quando indicado.

Quadro B.1 – Exemplos de comandos de voz

Função	**Português**	**Inglês**
Chamar o assistente	ok Fael	ok Fael
Chamar e comandar	ok Fael, anota tarefa comprar pão	ok Fael, add task buy bread
Criar tarefa	anota tarefa revisar o relatório	add task review the report
Listar tarefas	quais são minhas tarefas?	what are my tasks?
Concluir tarefa	terminei a tarefa 2	complete task 2
Lembrete relativo	me lembra daqui a 20 minutos de beber água	remind me in 20 minutes to drink water
Lembrete em data e hora (requer o modelo)	amanhã às 9 me lembra da reunião	tomorrow at 9 remind me of the meeting
Histórico	o que eu fiz ontem? o que concluí na semana passada?	what did I do yesterday? what did I do last week?
Abrir programa	abre o navegador; abrir a calculadora	open the browser; open the calculator
Hora e data	que horas são? que dia é hoje?	what time is it? what's the date?
Nova conversa	nova conversa	new conversation
Encerrar o modo contínuo	tchau	bye

Fonte: elaboração própria (2026).

APÊNDICE C – CONFIGURAÇÃO

O Quadro C.1 relaciona as variáveis de ambiente e as opções de linha de comando. A precedência é: preferência salva na tela de configurações, depois opção de linha de comando ou variável de ambiente e, por fim, valor padrão.

Quadro C.1 – Variáveis de ambiente e opções de linha de comando

Variável ou opção	**Padrão**	**Efeito**
`ANTHROPIC_API_KEY`	–	Ativa o Claude; sem ela, modelo local (Ollama) ou modo offline
`ROBOFAEL_CEREBRO`	auto	`auto` (Claude, depois Ollama, depois offline), `claude`, `ollama` ou `offline`
`ROBOFAEL_OLLAMA_URL`	`http://localhost:11434`	Servidor do Ollama
`ROBOFAEL_OLLAMA_MODELO`	`llama3.2:3b`	Modelo local (sem ele, o primeiro instalado que aceite ferramentas)
`ROBOFAEL_USUARIO`	Rafael	Nome usado na saudação e nas instruções do modelo
`ROBOFAEL_ASSISTENTE`	Robofael	Nome do assistente
`ROBOFAEL_FRASE_ATIVACAO`	ok Fael	Frase de ativação
`ROBOFAEL_ATIVACAO` ou `--sem-ativacao`	1 (ligada)	Liga ou desliga a escuta passiva
`ROBOFAEL_IDIOMA` ou `--idioma`	pt-BR	Idioma das respostas e da interface (pt-BR ou en-US)
`ROBOFAEL_MODELO`	`claude-opus-5-5`	Modelo do Claude
`ROBOFAEL_VOZ`	`pt-BR-AntonioNeural`	Voz neural padrão
`ROBOFAEL_DADOS`	`./dados` (instalado: `~/.local/share/robofael`)	Banco de dados (que pode ser movido nas configurações), perfil e gravações da voz
`ROBOFAEL_CACHE`	`./.cache` (instalado: `~/.cache/robofael`)	Quadros do avatar e falas sintetizadas
`ROBOFAEL_ASSETS`	`./assets`	Fotografias dos avatares
`ROBOFAEL_MODELOS`	`./modelos`	Modelo de reconhecimento local
`--avatar`	o salvo	Avatar inicial (crachá ou *selfie*)
`--continuo`	desligado	Volta a ouvir após cada resposta
`-v`	desligado	Registros detalhados (texto ouvido, pico e ganho)

Fonte: elaboração própria (2026).

Nota: os nomes anteriores à versão 3.5.0 (`EUROBO_*`) ainda são aceitos, com aviso de depreciação, quando a variável `ROBOFAEL_*` correspondente não está definida.

APÊNDICE D – PROCEDIMENTOS DE INSTALAÇÃO, EXECUÇÃO E VERIFICAÇÃO

Instalação pelo pacote (Ubuntu 24.04). Na pasta do projeto, executar `make deb` para gerar o pacote em `dist/`. Em seguida, instalá-lo com `sudo apt install ./dist/robofael_3.8.0_amd64.deb`. O gerenciador de pacotes instala as dependências de sistema, e o pacote cria o ambiente Python em `/opt/robofael` sem acessar a internet. O assistente é aberto pelo item Robofael do menu de aplicativos ou pelo comando `robofael`; a ferramenta de gravação da voz, pelo item correspondente ou pelo comando `robofael-gravar-voz`. Para remover, usar `sudo apt remove robofael`, que preserva os dados, ou `sudo apt purge robofael`, que também apaga a pasta do programa; os dados pessoais só são removidos manualmente.

Configurações de uso. O botão de engrenagem abre a tela de configurações. Além de idioma, avatar, voz e ativação por voz, ela permite escolher o microfone; marcar as opções de abrir o assistente ao entrar no computador e de usar o comando `robofael` em qualquer terminal (no pacote, o comando já existe); ver o modelo de linguagem em uso e os modelos compatíveis do Claude e do Ollama; e, na seção de memória, mudar o banco de pasta ou fazer um backup, operações que valem na hora. Para restaurar um backup, basta fechar o assistente e copiar o arquivo do backup sobre o `memoria.db`. Como o assistente aberto pelo login não lê o `~/.bashrc`, a chave do Claude, nesse caso, deve ser definida em `~/.config/environment.d/`.

Execução em modo de desenvolvimento. Executar `make instalar` para criar o ambiente com as dependências de desenvolvimento, `make baixar-modelo-voz` para obter o modelo de reconhecimento local e `make executar` para abrir o assistente. Opções podem ser passadas com `make executar ARGS="--idioma en-US -v"`. A gravação da voz é feita com `make gravar-voz`.

Verificação. O comando `make verificar` executa a análise estática, a verificação de tipos e os 643 testes; `make testar-rapido` omite os testes de interface e de renderização; `make cobertura` gera o relatório de cobertura; `make integridade` verifica a cadeia de resumos do histórico; e `make testar-deb` constrói o pacote e o testa em uma pasta temporária.

Geração deste documento. O texto desta monografia está em `docs/monografia`, em LaTeX, e é compilado com `make monografia`, que produz o arquivo `docs/monografia/monografia.pdf`.